# Cascaded Convolutional Neural Network for Solar Panel Defect Classification

21CSC402P-PROJECT

*Submitted by*

Divyanshi Jain [RA2311026010639]
Syed Mohammed Ghouse S [RA2311026010643]
Purvee Singh [RA2311026010645]

*Under the Guidance of*

Dr. Illakiya T
(Assistant Professor, Department of Computational Intelligence)

*in partial fulfillment of the requirements for the degree of*

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in Artificial Intelligence and Machine Learning




DEPARTMENT OF COMPUTATIONAL INTELLIGENCE COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203
NOVEMBER 2026

Department of Computational Intelligence

**SRM Institute of Science & Technology**

**Own Work Declaration Form**

To be completed by the student for all assessments

| | |
|---|---|
| **Degree/ Course** | **: B. Tech Computer Science with specialization in AI/ML** |
| **Student Name** | **: Divyanshi Jain, Syed Mohammed Ghouse S, Purvee Singh** |
| **Registration Number** | **: RA2311026010639, RA2311026010643, RA2311026010645** |
| **Title of Work** | **: Cascaded Convolutional Neural Network for Solar Panel Defect Classification** |

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that I have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc.)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g.fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook /University website

We understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

**DECLARATION:**

We are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is our own work, except were indicated by referring, and that we have followed the good academic practices noted above.

Divyanshi Jain, RA2311026010639
Syed Mohammed Ghouse S, RA2311026010643
Purvee Singh, RA2311026010645

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSC402P - Project report titled “Cascaded Convolutional Neural Network for Solar Panel Defect Classification” is the bonafide work of “Divyanshi Jain [RA2311026010639], Syed Mohammed Ghouse S [RA2311026010643], Purvee Singh [RA2311026010645]” who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

**SIGNATURE**

**DR. ILLAKIYA T**
**SUPERVISOR**
ASSISTANT PROFESSOR
DEPARTMENT OF
COMPUTATIONAL INTELLIGENCE

**SIGNATURE**

**DR. R. ANNIE UTHRA**
**PROFESSOR &HEAD**
DEPARTMENT OF
COMPUTATIONAL INTELLIGENCE

## ACKNOWLEDGEMENT

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr. C. Lakshmi,** Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. R. Annie Uthra**, Professor & Department of Computational Intelligence, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head, and Panel Members Department of Computational Intelligence, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, Dr. Illakiya T, Department of Computational Intelligence, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, Dr. Illakiya T, Department of Computational Intelligence, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under their mentorship, and for the freedom and support to explore the topics of our interest.

We sincerely thank all the staff members of Department of Computational Intelligence, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement.

Divyanshi Jain
Syed Mohammed Ghouse S
Purvee Singh

# ABSTRACT

Automated inspection of large photovoltaic (PV) installations is essential for sustained power output and safety, but manual thermal surveys do not scale and flat single-stage classifiers struggle with the severe class imbalance and look-alike faults of the InfraredSolarModules benchmark. This project presents Cascaded-SolarNet, a three-stage cascade of five small convolutional neural networks that classifies the 12 classes of the benchmark from single-modality infrared images of only 24 x 40 pixels. Stage 1 separates healthy from defective panels and lets healthy panels exit early; Stage 2 assigns a defect to a fault family; and Stage 3 applies one specialist network per family to name the exact fault. The system was implemented in PyTorch and evaluated on a fixed stratified held-out split of 3,000 images that is never used for training, early stopping or model selection. Removing brightness, contrast and noise augmentation, which destroys the temperature signal of thermal images, raised the from-scratch cascade from 61.7% to 79.8% accuracy, and fine-tuning an ImageNet-pretrained EfficientNet-B0 at every stage reached 83.2% accuracy, macro-F1 0.700 and 95.8% on the healthy-versus-defect decision. A flat classifier with the same backbone is essentially tied (82.6%), so the cascade did not show a clear accuracy advantage, and the family-routing stage remains the largest source of errors. Published 12-class results of up to 93.9% rely on cross-validation and pretrained features and are not directly comparable with a fixed hold-out; the report presents this comparison and its limits. Code, trained models, architecture diagrams and interactive demonstrations are provided.

**Index Terms:** Solar panel defect classification, infrared thermography, cascaded convolutional neural network, early exit, hierarchical classification, transfer learning, class imbalance.

# TABLE OF CONTENTS


| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| | **ABSTRACT** | **vi** |
| | **TABLE OF CONTENTS** | **vii** |
| | **LIST OF FIGURES** | **xi** |
| | **LIST OF TABLES** | **xiii** |
| | **ABBREVIATIONS** | **xv** |
| **1** | **INTRODUCTION** | **1** |
| 1.1 | Introduction to Solar Panel Defect Classification | 1 |
| 1.2 | Problem Statement and Description | 1 |
| 1.3 | Motivation | 2 |
| 1.4 | Sustainable Development Goal of the Project | 3 |
| **2** | **LITERATURE SURVEY** | **4** |
| 2.1 | Overview of PV Defect Detection and Classification Methods | 4 |
| 2.2 | Deep Learning Approaches for PV Defect Analysis | 4 |
| 2.2.1 | The InfraredSolarModules Benchmark and Its Baselines | 4 |
| 2.2.2 | Cascaded and Dual-Sensor Approaches | 4 |
| 2.2.3 | Lightweight Networks | 5 |
| 2.2.4 | Detection and Segmentation Pipelines | 5 |
| 2.2.5 | Transfer Learning, Ensembles and Feature Selection | 5 |
| 2.3 | Limitations of Existing Methods | 6 |
| 2.4 | Research Gaps and Contributions | 6 |
| 2.5 | Research Objectives | 6 |
| 2.6 | Plan of Action | 7 |
| **3** | **PROBLEM FORMULATION** | **8** |

| | | |
|---|---|---|
| 3.1 | Formal Problem Definition | 8 |
| 3.2 | Decomposition into Sub-Problems | 8 |
| 3.2.1 | Healthy Versus Defective (Triage) | 8 |
| 3.2.2 | Fault Family | 8 |
| 3.2.3 | Fine-Grained Subtype | 9 |
| 3.3 | Constraints and Assumptions | 9 |
| 3.3.1 | Dataset | 9 |
| 3.3.2 | Input Modality and Image Size | 9 |
| 3.3.3 | Whole-Image Classification | 9 |
| 3.3.4 | Evaluation Protocol | 10 |
| 3.4 | Why the Problem Is Non-Trivial | 10 |
| 3.4.1 | Class Imbalance | 10 |
| 3.4.2 | Look-Alike Faults | 10 |
| 3.4.3 | Error Propagation | 10 |
| 3.5 | Problem Statement Summary | 10 |
| **4** | **PROPOSED SOLUTION** | **11** |
| 4.1 | Hierarchical Three-Stage Cascade | 11 |
| 4.2 | Early Exit for Healthy Panels | 11 |
| 4.3 | Fault-Family Grouping | 11 |
| 4.4 | Two Backbones: Lightweight From Scratch and ImageNet-Pretrained | 11 |
| 4.5 | Single-Modality, Whole-Image Pipeline | 12 |
| 4.6 | Summary of Contributions | 12 |
| **5** | **ARCHITECTURE AND IMPLEMENTATION** | **13** |
| 5.1 | Overview of the Implemented System | 13 |
| 5.2 | Data Pipeline | 14 |
| 5.2.1 | Dataset and Class Distribution | 14 |

| | | |
|---|---|---|
| 5.2.2 | Split and Normalisation | 15 |
| 5.2.3 | Augmentation | 15 |
| 5.2.4 | Handling Class Imbalance | 15 |
| 5.3 | Building Blocks and Their Mathematics | 16 |
| 5.3.1 | Convolution | 16 |
| 5.3.2 | Batch Normalisation | 16 |
| 5.3.3 | ReLU and Max Pooling | 16 |
| 5.3.4 | Global Average Pooling and Classification Head | 17 |
| 5.3.5 | Loss Function | 17 |
| 5.3.6 | Counting Parameters | 17 |
| 5.4 | Stage 1: CNN-1, Binary Triage | 17 |
| 5.5 | Stage 2: CNN-2, Fault-Family Classifier | 18 |
| 5.6 | Stage 3: Fine-Grained Specialists | 19 |
| 5.7 | Pretrained Variant | 21 |
| 5.8 | Training Procedure | 21 |
| 5.9 | Cascade Inference and Cost Analysis | 22 |
| 5.10 | Evaluation Tools and Demonstrations | 22 |
| 5.11 | Testing and Reproducibility | 23 |
| 5.12 | Chapter Summary | 24 |
| **6** | **RESULTS AND ANALYSIS** | **25** |
| 6.1 | Experimental Setup | 25 |
| 6.1.1 | Dataset and Protocol | 25 |
| 6.1.2 | Implementation and Training Configuration | 25 |
| 6.1.3 | Evaluation Metrics | 25 |
| 6.1.4 | Configurations | 26 |
| 6.2 | Overall Performance | 26 |

| | | |
|---|---|---|
| 6.2.1 | Test Results | 26 |
| 6.2.2 | Per-Network Validation Results and Training Behaviour | 27 |
| 6.3 | Ablation Study | 28 |
| 6.3.1 | Photometric Augmentation | 28 |
| 6.3.2 | Input Size and Network Capacity | 29 |
| 6.3.3 | Balanced Sampling and Label Smoothing | 29 |
| 6.3.4 | Inference-Time Variants | 29 |
| 6.3.5 | Fault-Family Grouping | 29 |
| 6.3.6 | ImageNet Pretraining | 30 |
| 6.3.7 | Summary of the Ablations | 30 |
| 6.4 | Error Analysis by Stage | 30 |
| 6.5 | Per-Class Performance and Confusion | 31 |
| 6.5.1 | Per-Class F1 | 31 |
| 6.5.2 | Confusion Matrices | 32 |
| 6.6 | Comparison with Published Work | 34 |
| 6.7 | Data Overlap Analysis | 36 |
| 6.8 | Discussion and Limitations | 36 |
| 6.8.1 | Interpretation of Findings | 36 |
| 6.8.2 | Limitations | 36 |
| 6.8.3 | Threats to Validity | 37 |
| 6.9 | Chapter Summary | 37 |
| **7** | **CONCLUSION AND FUTURE SCOPE** | **38** |
| 7.1 | Conclusion | 38 |
| 7.2 | Future Scope | 38 |
| | **REFERENCES** | **39** |
| **A** | **CONFIGURATION FILES** | **41** |

| | | |
|---|---|---|
| **B** | **KEY SOURCE CODE** | **43** |
| **C** | **RUNNING THE DEMONSTRATIONS** | **46** |
| **D** | **REPOSITORY STRUCTURE** | **48** |

# LIST OF FIGURES

| FIGURE NO. | TITLE | PAGE NO. |
|---|---|---|
| 1 | 1. The implemented three-stage Cascaded-SolarNet architecture | 13 |
| 2 | 2. Class distribution of the InfraredSolarModules dataset (logarithmic axis) | 14 |
| 3 | 3. One test image per class, shown with the inferno colour map | 15 |
| 4 | 4. Stage 1: CNN-1, binary triage (layers and feature-map sizes) | 18 |
| 5 | 5. Stage 2: CNN-2, fault-family classifier | 19 |

| FIGURE NO. | TITLE | PAGE NO. |
|---|---|---|
| 6 | 6. Stage 3: the three specialists and their classes | 20 |
| 7 | 7. Output of the live demonstration: each stage's decision and confidence for Hot-Spot test images | 23 |
| 8 | 8. Cascade test accuracy and macro-F1 by configuration (dashed line: from-scratch flat CNN) | 27 |
| 9 | 9. Best validation macro-F1 of each stage network, from scratch and pretrained | 27 |
| 10 | 10. Validation macro-F1 and training loss of the five stage networks | 28 |
| 11 | 11. Where the cascade makes its errors, by configuration | 31 |
| 12 | 12. Per-class F1 on the test set | 32 |
| 13 | 13. Confusion matrix of the pretrained cascade on the test set (row-normalised, counts inside cells) | 33 |
| 14 | 14. Confusion matrix of the pretrained flat CNN on the test set | 34 |
| 15 | 15. Example output of the command-line demonstration for random test images (green: correct, red: wrong) | 47 |

# LIST OF TABLES


| TABLE NO. | TITLE | PAGE NO. |
|---|---|---|
| 2.1 | Summary of the reviewed studies | 5 |
| 2.2 | Research gaps and the contributions that address them | 6 |
| 3.1 | Class distribution and stratified split (seed 42) | 9 |
| 4.1 | Contributions and the gaps they address | 12 |
| 5.1 | Layer-by-layer structure of CNN-1 (from-scratch variant) | 18 |
| 5.2 | Layer-by-layer structure of CNN-2 (from-scratch variant) | 19 |
| 5.3 | Parameter counts of the five stage networks | 20 |
| 5.4 | Architecture summary of the five networks (from-scratch variant) | 20 |
| 5.5 | Training data of each stage network | 21 |
| 5.6 | Automated tests | 23 |
| 6.1 | Training configuration of the evaluated models | 25 |
| 6.2 | Definition of the evaluated configurations | 26 |
| 6.3 | Test results by configuration | 26 |
| 6.4 | Effect of photometric augmentation (cascade, 40 x 24 input) | 28 |
| 6.5 | Effect of input size and capacity (cascade, flips only) | 29 |
| 6.6 | Inference-time variants on the improved cascade (no retraining) | 29 |
| 6.7 | Semantic versus data-driven family grouping (from-scratch cascade) | 30 |
| 6.8 | Test errors per stage (out of 3,000 images) | 30 |

| TABLE NO. | TITLE | PAGE NO. |
|---|---|---|
| 6.9 | Per-class F1 on the test set | 31 |
| 6.10 | Most frequent confusions of the pretrained cascade | 34 |
| 6.11 | Published results on InfraredSolarModules and results of this project | 35 |
| 6.12 | Similarity between test and training images (raw pixels) | 36 |
| D.1 | Layout of the project repository | 48 |

# ABBREVIATIONS

| ABBREVIATION | FULL FORM |
|---|---|
| AI | Artificial Intelligence |
| AMP | Automatic Mixed Precision |
| BN | Batch Normalisation |
| CNN | Convolutional Neural Network |
| CV | Cross-Validation |
| EL | Electroluminescence |
| F1 | Harmonic mean of precision and recall |
| FC | Fully Connected (layer) |
| GAP | Global Average Pooling |
| GPU | Graphics Processing Unit |
| IR | Infrared |
| NCA | Neighbourhood Component Analysis |
| PV | Photovoltaic |
| ReLU | Rectified Linear Unit |
| RF | Random Forest |
| SDG | Sustainable Development Goal |
| SVM | Support Vector Machine |
| TTA | Test-Time Augmentation |
| UAV | Unmanned Aerial Vehicle |
| ViT | Vision Transformer |

# CHAPTER 1

# INTRODUCTION

Solar farms contain thousands of photovoltaic modules that develop defects over time, including hot spots, cracks, soiling, shadowing and diode failures. Each of these reduces power output, and some, such as undetected hot spots, create a fire risk. Manual thermal inspection is slow, expensive, labour-intensive and error-prone, and it cannot scale to utility-grade installations. Deep learning applied to drone-captured infrared (IR) imagery offers an automated alternative: an unmanned aerial vehicle with a thermal camera can image thousands of modules in one flight, and a trained classifier can label each module in milliseconds.

This project builds and evaluates such a classifier on the InfraredSolarModules benchmark, a public dataset of 20,000 IR images of 24 x 40 pixels with 12 classes (one healthy class and eleven fault types). Instead of a single network that chooses directly among all twelve classes, the project proposes a cascade of small networks that answers three easier questions in turn.

## 1.1 Introduction to Solar Panel Defect Classification

In an infrared image brightness encodes temperature. A healthy module heats fairly uniformly during operation, whereas a defect produces a localised temperature anomaly: a hot spot, a cold region or a characteristic thermal pattern. Classifying these patterns is therefore a pattern-recognition problem on very small images, where fine texture is absent and the position, size and shape of heat concentrations carry the information. Typical defect families are electrical faults (cells and bypass diodes), environmental conditions (soiling, shadows, vegetation) and structural damage (cracks, offline modules).

The task is difficult for three reasons. The classes are strongly imbalanced, because half of the images show healthy modules. Several fault types look alike at low resolution. And the system has to be cheap enough to run on drone hardware. These properties motivate a design that removes the dominant class first and then specialises.

## 1.2 Problem Statement and Description

Most existing CNN approaches use a flat, single-stage classifier that must choose directly among all classes. This design has two weaknesses. First, visually similar fault types produce overlapping features, so the network confuses them. Second, healthy panels vastly outnumber faulty ones, which biases the network towards the majority class and hurts the rare faults. The

literature reviewed in this project reports that accuracy falls from about 73% for binary tasks to about 68% for multi-class tasks across eight architectures, and that single-stage results on this benchmark stop at about 93%.

The problem addressed here is to classify a single-modality 24 x 40 pixel IR image of a PV module into one of 12 classes accurately, with a model small enough for edge use, and to understand where the errors of the proposed design come from. The literature survey identified six gaps:

- **G1.** Dual-sensor approaches are impractical. Models reaching 98% to 99% accuracy need paired thermal and visual data that a single-camera drone cannot supply.
- **G2.** Scaling the model gives negligible gains. Heavier backbones add little accuracy over lightweight ones.
- **G3.** Bounding-box detectors split thin defects. Detection models mislocalise fine crack boundaries.
- **G4.** Naive cascades add latency. Detection-plus-classification pipelines are too slow for real-time use.
- **G5.** Accuracy drops as the number of classes grows, from roughly 73% for binary to 68% for multi-class classification.
- **G6.** Similar defect types are systematically confused, and the confusion persists in flat classifiers.

## 1.3 Motivation

The practical cost of misclassification is concrete. A missed hot spot can become a fire; a false alarm sends a maintenance crew to a healthy module. Because most modules are healthy, a classifier that wastes effort on them is both slow and biased. A hierarchical design addresses this in a natural way: a small first network handles the easy and most frequent decision and passes only suspicious modules on, and later networks can then spend their capacity on the genuinely hard distinctions.

A second motivation is scientific. Cascades are often assumed to beat flat classifiers, but the literature offers few controlled comparisons on a small thermal benchmark. This project measures that claim with identical building blocks, identical data splits and a stage-by-stage attribution of errors, and reports the result honestly, including where the cascade does not help.

## 1.4 Sustainable Development Goal of the Project

The project contributes most directly to SDG 7, Affordable and Clean Energy. Solar installations that are inspected faster and more cheaply lose less energy to undetected defects, which improves the return on clean-energy investment. It also supports SDG 13, Climate Action, because every percentage point of recovered solar output displaces fossil generation, and SDG 9, Industry, Innovation and Infrastructure, through low-cost automated inspection that can run on small drones.

# CHAPTER 2

# LITERATURE SURVEY

This chapter reviews deep-learning approaches to PV defect detection and classification, with emphasis on infrared imagery and on the InfraredSolarModules benchmark, and derives the research gaps and objectives of the project. The studies were re-checked against their primary sources while preparing the final report; where a result had been attributed to the wrong paper in the first survey, it has been corrected below.

## 2.1 Overview of PV Defect Detection and Classification Methods

Automated analysis of PV imagery has moved through several generations. Early work extracted handcrafted thermal features and classified them with classical learners such as random forests. The second generation applied convolutional networks end to end, from small custom CNNs to ImageNet-pretrained VGG, ResNet and EfficientNet backbones. A third thread moved to object detection and segmentation (Faster R-CNN, Cascade R-CNN, VarifocalNet, YOLO) to localise defects. The most recent work explores cascades, ensembles, transfer learning with heavy augmentation and feature-selection pipelines, usually at the cost of extra sensors or computation.

## 2.2 Deep Learning Approaches for PV Defect Analysis

### 2.2.1 The InfraredSolarModules Benchmark and Its Baselines

Millendorf, Obropta and Vadhavkar of Raptor Maps released the InfraredSolarModules dataset at the ICLR 2020 Workshop on AI for Earth Sciences [1]. The dataset paper describes the data and lists benchmark classification results as a goal, but it reports no model accuracies. Baselines on the dataset come from later papers: a random forest at 84.0% [4], a custom CNN with augmentation at 92.5% for binary anomaly detection and 78.85% for eight selected classes [3], and an EfficientNet-B0 feature pipeline with an SVM at 93.93% for all 12 classes under 10-fold cross-validation [2].

### 2.2.2 Cascaded and Dual-Sensor Approaches

Pathak and Patil proposed a Binary Cascaded CNN that first separates healthy from faulty modules and then identifies the fault, fusing thermal images with co-registered visible-spectrum images [5]. It reports 99.8% precision and 98.5% accuracy, the highest in the survey,

but it needs paired dual-sensor data and substantial computation, which a single-camera drone does not provide. Barraz et al. describe a cascading ensemble with a voting CNN reaching 88% on InfraredSolarModules, as cited in [4].

### 2.2.3 Lightweight Networks

PV-faultNet benchmarked architectures from compact CNNs to Siamese and multi-spectral models and found that a carefully designed compact CNN reaches 93% accuracy at 8 ms per image, with larger backbones adding little [6]. The authors conclude that the bottleneck is the low information content of 24 x 40 pixel images and not model capacity, which supports small per-stage networks.

### 2.2.4 Detection and Segmentation Pipelines

Detection-based work extends VarifocalNet and compares it with Cascade R-CNN and YOLOv7 for localising defects in drone IR images [7]. Both detectors struggle with thin linear defects such as cracks, producing fragmented or offset boxes. Otamendi et al. combine Faster R-CNN with an EfficientNet classifier on electroluminescence images [8]; it is precise but needs powered panels imaged in darkness and adds seconds of latency. These results motivate whole-image classification for the low-resolution IR domain.

### 2.2.5 Transfer Learning, Ensembles and Feature Selection

A field-adapted ResNet-50 with 8x augmentation reaches 98.59% on field data but is heavy for edge use. CNN ensembles with Bacterial Foraging Optimisation for feature selection reach 98.75% with VGG-16 but must run five backbones. Tella et al. benchmarked eight CNN and ViT architectures on the ELPV benchmark and found an average of about 73% for binary and 68% for multi-class classification [9]. A four-class RGB study (normal, cracked, dusty, shadowed) reaches 96% to 98% but its coarse labels hide the real difficulty. Table 2.1 summarises the studies.

**Table 2.1: Summary of the reviewed studies**

| # | Study | Approach | Key result | Limitation |
|---|---|---|---|---|
| 1 | Millendorf et al., 2020 | Dataset paper (InfraredSolarModules) | No model results reported | Baselines come from later work |
| 2 | Pathak and Patil, 2025 | Binary cascaded CNN with thermal-visual fusion | 99.8% precision, 98.5% accuracy | Needs paired thermal and visual data |
| 3 | PV-faultNet, 2024 | Compact CNN versus Siamese and multi-spectral | 93% at 8 ms per image | Larger backbones gain little |
| 4 | Improved VarifocalNet, 2024 | Cascade R-CNN, YOLOv7 comparison | High coarse detection confidence | Mislocalises thin cracks |
| 5 | Otamendi et al., 2021 | Faster R-CNN + EfficientNet | High precision | Too slow for real-time |

| # | Study | Approach | Key result | Limitation |
|---|---|---|---|---|
| | | on EL images | | drones |
| 6 | Field-adapted ResNet-50 | Transfer learning with 8x augmentation | 98.59% on field data | High resource demand |
| 7 | Tella et al., 2025 | 8 CNN/ViT models on ELPV | About 68% multi-class vs 73% binary | Large binary to multi-class drop |
| 8 | CNN on RGB images | 4-class normal/cracked/dusty/shadowed | 96% to 98% | Coarse classes only |
| 9 | CNN + Bacterial Foraging Optimisation | Five pretrained CNNs with feature selection | VGG-16 + BFO: 98.75% | Extra tuning, five backbones |
| 10 | Duranay, 2023 | EfficientNet-B0 features, NCA, SVM | 93.93% on 12 classes (10-fold CV) | Cross-validation, pretrained features |

## 2.3 Limitations of Existing Methods

Four limitations recur across the reviewed work. The highest accuracies depend on information a drone does not have (paired visible images, powered-panel electroluminescence) or on heavy computation (ensembles, large backbones). Detection pipelines add failure modes on thin defects. Results are reported under different protocols, mostly cross-validation, which makes them hard to compare, and near-duplicate crops in the dataset can inflate cross-validated scores. Finally, almost no study reports where a hierarchical model loses accuracy, so the benefit of cascading remains unmeasured.

## 2.4 Research Gaps and Contributions

Table 2.2 maps each gap to the contribution of this project.

**Table 2.2: Research gaps and the contributions that address them**

| Gap | Research gap | Contribution |
|---|---|---|
| G1 | Dual-sensor approaches are impractical | C4: single-modality design using only single-camera IR images |
| G2 | Scaling the model gives negligible gains | C1: five small per-stage networks instead of one large model |
| G3 | Bounding boxes fragment thin defects | C5: whole-image classification with no localisation step |
| G4 | Naive cascades add latency | C2: early-exit binary front end for healthy panels |
| G5 | Accuracy drops as classes grow | C1: hierarchical decomposition into 2-, 3- and 2-to-6-way decisions |
| G6 | Similar defects are confused | C3: family grouping before fine classification, semantic and data-driven |

## 2.5 Research Objectives

- **O1.** To design a three-stage cascade of small CNNs for 12-class IR solar module classification from a single sensor.

- **O2.** To implement the system reproducibly in PyTorch with a fixed stratified split, tests and published code.
- **O3.** To measure, with controlled ablations, which design choices (augmentation, input size, capacity, grouping, pretraining) matter.
- **O4.** To attribute the cascade's errors to the stage that caused them and to compare it with a flat classifier and with published results.

## 2.6 Plan of Action

The work followed six steps: (1) literature survey and gap analysis; (2) dataset preparation with a stratified 70/15/15 split; (3) implementation of the five-network cascade and a flat baseline; (4) controlled ablations of augmentation, input size, network capacity, family grouping and inference variants; (5) fine-tuning of an ImageNet-pretrained backbone at every stage; and (6) evaluation, error attribution, comparison with published work, architecture diagrams, demonstrations and this report.

# CHAPTER 3

# PROBLEM FORMULATION

This chapter defines the classification problem precisely, decomposes it into sub-problems, states the constraints under which the system is built and explains why a naive solution is not enough.

## 3.1 Formal Problem Definition

Let x be a single-channel infrared image of one PV module with 40 rows and 24 columns, x in R^(40 x 24) with values scaled to [0, 1]. Let the label set be Y = {y0, y1, ..., y11}, where y0 is No-Anomaly and y1 to y11 are the fault classes. The goal is to learn a function f that maps x to a label so that the expected accuracy and the macro-averaged F1 score over Y are maximised on unseen images, subject to the constraints of Section 3.3.

The label set carries a hierarchy: Y = {y0} U F_E U F_V U F_S, where the families F_E (Electrical), F_V (Environmental) and F_S (Structural) partition the eleven faults. This structure is exploited by the cascade: f(x) = g_fam(x)(x) if the triage function h(x) predicts a defect, and y0 otherwise.

## 3.2 Decomposition into Sub-Problems

## 3.2.1 Healthy Versus Defective (Triage)

The first sub-problem is the binary function h(x) in {healthy, defect}. It is the easiest decision in the system and the one that removes the dominant class. An error here is irreversible: a missed defect is never seen by later stages, and a false alarm only costs computation and a wrong label.

## 3.2.2 Fault Family

For images judged defective, the second sub-problem is the three-way function that selects a family. Faults that share a family should look similar to each other and different from other families, so this decision should be much easier than an eleven-way choice. Section 6 tests how well this holds.

### 3.2.3 Fine-Grained Subtype

Within a family, a specialist function chooses the exact fault: six classes for Electrical, three for Environmental and two for Structural. Each specialist is trained only on its own family and can spend all its capacity on the distinctions inside it.

## 3.3 Constraints and Assumptions

### 3.3.1 Dataset

The InfraredSolarModules dataset contains 20,000 images from real drone flights, 24 pixels wide and 40 pixels tall. The classes and their distribution in the stratified split used throughout this report are given in Table 3.1.

**Table 3.1: Class distribution and stratified split (seed 42)**

| Class | Family | Total | Train | Validation | Test |
|---|---|---|---|---|---|
| No-Anomaly | Healthy | 10000 | 7000 | 1500 | 1500 |
| Cell | Electrical | 1877 | 1314 | 282 | 281 |
| Cell-Multi | Electrical | 1288 | 902 | 193 | 193 |
| Diode | Electrical | 1499 | 1049 | 225 | 225 |
| Diode-Multi | Electrical | 175 | 123 | 26 | 26 |
| Hot-Spot | Electrical | 249 | 174 | 38 | 37 |
| Hot-Spot-Multi | Electrical | 246 | 172 | 37 | 37 |
| Soiling | Environmental | 204 | 143 | 30 | 31 |
| Shadowing | Environmental | 1056 | 739 | 158 | 159 |
| Vegetation | Environmental | 1639 | 1147 | 246 | 246 |
| Cracking | Structural | 940 | 658 | 141 | 141 |
| Offline-Module | Structural | 827 | 579 | 124 | 124 |
| **Total** | | **20000** | **14000** | **3000** | **3000** |

### 3.3.2 Input Modality and Image Size

Only single-channel IR images are used, because a single-camera drone cannot supply paired visible images. The images are tiny, so networks must recognise where heat is concentrated and what shape it takes rather than rely on texture.

### 3.3.3 Whole-Image Classification

The system classifies the whole module image and does not localise defects. Detection pipelines were shown in the literature to fragment thin defects, and the dataset provides one label per crop and no boxes.

### 3.3.4 Evaluation Protocol

All results are measured on one fixed, stratified held-out test split of 3,000 images. The test images are never used for training, early stopping, normalisation statistics or model selection; all tuning uses a separate 3,000-image validation split.

## 3.4 Why the Problem Is Non-Trivial

### 3.4.1 Class Imbalance

Half of the images are healthy, and the rarest classes have 175 to 249 images in total (123 to 174 for training). A classifier that always answers No-Anomaly scores 50% accuracy and learns nothing about faults.

### 3.4.2 Look-Alike Faults

Several classes differ only in how many cells or diodes are affected (Cell and Cell-Multi, Diode and Diode-Multi, Hot-Spot and Hot-Spot-Multi), and at 24 x 40 pixels these differences are subtle.

### 3.4.3 Error Propagation

In a cascade, a wrong decision at an early stage cannot be corrected later. The overall accuracy of a defective image is roughly the product of the accuracies of the stages it passes through, so the early stages must be very accurate. Whether a cascade nevertheless beats a flat classifier is an empirical question, which this project answers by measurement.

## 3.5 Problem Statement Summary

Given single-modality 40 x 24 IR images of PV modules, build a small-network system that classifies them into 12 classes, evaluate it on a clean held-out split, attribute its errors to individual stages and compare it with a flat classifier and with published results.

# CHAPTER 4

# PROPOSED SOLUTION

This chapter describes the design decisions of Cascaded-SolarNet and relates each one to a gap identified in Chapter 2. The detailed architecture and implementation follow in Chapter 5.

## 4.1 Hierarchical Three-Stage Cascade

Cascaded-SolarNet replaces one large 12-way classifier with five small specialist CNNs arranged in three stages. Each network answers one small, well-defined question, and the answer of one stage decides which network runs next. A healthy panel passes through only one network; a defective panel passes through exactly three: the triage network, the family network and one specialist.

## 4.2 Early Exit for Healthy Panels

Stage 1 is a binary classifier. When it predicts No-Anomaly the pipeline outputs that label and stops. Because healthy panels are the large majority, this removes the dominant class from all later networks, handles class imbalance by structure instead of relying only on loss re-weighting, and in principle saves computation on most images.

## 4.3 Fault-Family Grouping

Stage 2 routes each defect to one of three families. The semantic grouping follows the physical origin of the faults: Electrical (Cell, Cell-Multi, Diode, Diode-Multi, Hot-Spot, Hot-Spot-Multi), Environmental (Soiling, Shadowing, Vegetation) and Structural (Cracking, Offline-Module). As an alternative, a data-driven grouping was derived by testing every partition of the eleven faults into three families of at least three classes and choosing the one that minimises the confusions of a trained flat network on the validation split: group A (Cell, Cell-Multi, Soiling, Vegetation, Cracking), group B (Diode, Diode-Multi, Offline-Module) and group C (Hot-Spot, Hot-Spot-Multi, Shadowing). This reduced cross-family confusions on the validation split from 264 to 87.

## 4.4 Two Backbones: Lightweight From Scratch and ImageNet-Pretrained

Every stage network is available in two variants that share the cascade logic. The from-scratch

variant uses small convolutional blocks and has no external data. The pretrained variant uses an ImageNet-pretrained EfficientNet-B0 with a new classification head and is fine-tuned on the stage's data. The comparison of the two variants quantifies what pretraining adds on this tiny thermal domain.

## 4.5 Single-Modality, Whole-Image Pipeline

The system takes only a single-channel IR image and outputs a label for the whole module. It needs no second sensor, no alignment step and no localisation, which removes the failure modes G1 and G3.

## 4.6 Summary of Contributions

**Table 4.1: Contributions and the gaps they address**

| Contribution | Description | Gaps addressed |
|---|---|---|
| C1 | Hierarchical three-stage cascade in place of a flat 12-class classifier | G2, G5 |
| C2 | Early-exit binary front end that removes the dominant No-Anomaly class | G4, imbalance |
| C3 | Fault-family grouping (semantic and data-driven) before fine classification | G6 |
| C4 | Single-modality design using only single-camera IR images | G1 |
| C5 | Pure classification pipeline without localisation or bounding boxes | G3 |
| C6 | Controlled ablations and per-stage error attribution, with a leak-free evaluation protocol | Comparability |

# CHAPTER 5

# ARCHITECTURE AND IMPLEMENTATION

## 5.1 Overview of the Implemented System

Figure 1 shows the implemented cascade. The system is implemented in Python 3.12 with PyTorch 2.11 and torchvision, scikit-learn for the metrics and Matplotlib for plots. Training and evaluation ran on an NVIDIA GeForce RTX 5060 Laptop GPU with 8 GB of memory. The code is organised as a Python package (solarnet) with separate modules for the class hierarchy (taxonomy), data handling (data), the models, shared helpers (common), training (train) and evaluation (evaluate), plus configuration files, scripts for the baselines, cross-validation, figure and report generation, and a test suite. Appendix D lists the repository layout.

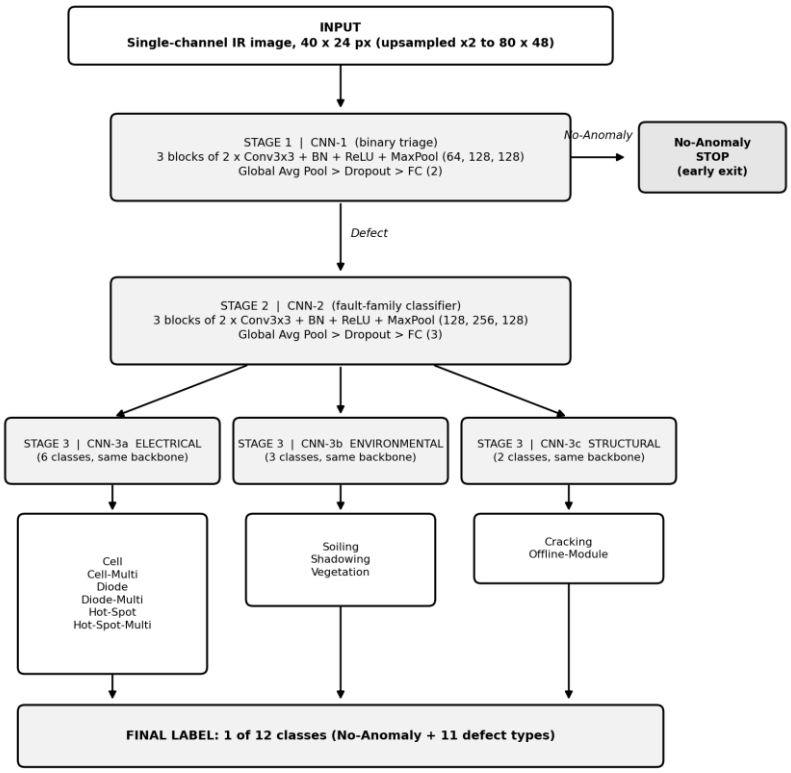



**Fig. 1. The implemented three-stage Cascaded-SolarNet architecture.**

An image is processed as follows. It is normalised and passed to CNN-1. If CNN-1 predicts No-Anomaly the pipeline stops. Otherwise CNN-2 predicts a family and the image is passed

to the specialist of that family, whose prediction is converted to a global class index by adding the family's class offset. Every network is trained independently on ground-truth routing, and the cascade is assembled only at evaluation time, so the error of a stage is measured exactly as it would occur in deployment.

## 5.2 Data Pipeline

### 5.2.1 Dataset and Class Distribution

The InfraredSolarModules dataset has 20,000 grayscale images of 24 pixels wide and 40 pixels tall. Figure 2 shows the class distribution on a logarithmic axis: half of the images are healthy, and the four rarest classes (Diode-Multi, Soiling, Hot-Spot-Multi and Hot-Spot) have between 175 and 249 images each. Figure 3 shows one test image per class with a thermal colour map. Even at this size the healthy panel is nearly uniform, while faults show localised bright regions whose position and extent differ by class.

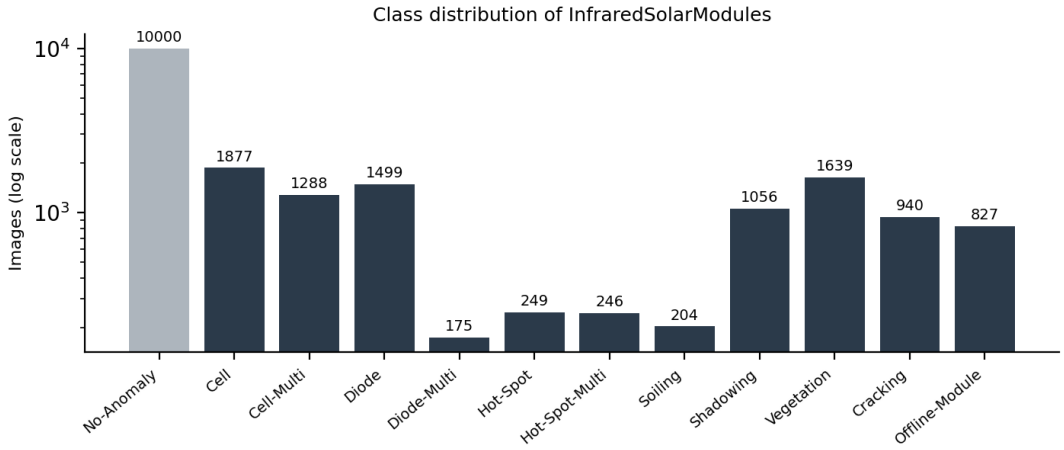



**Fig. 2. Class distribution of the InfraredSolarModules dataset (logarithmic axis).**

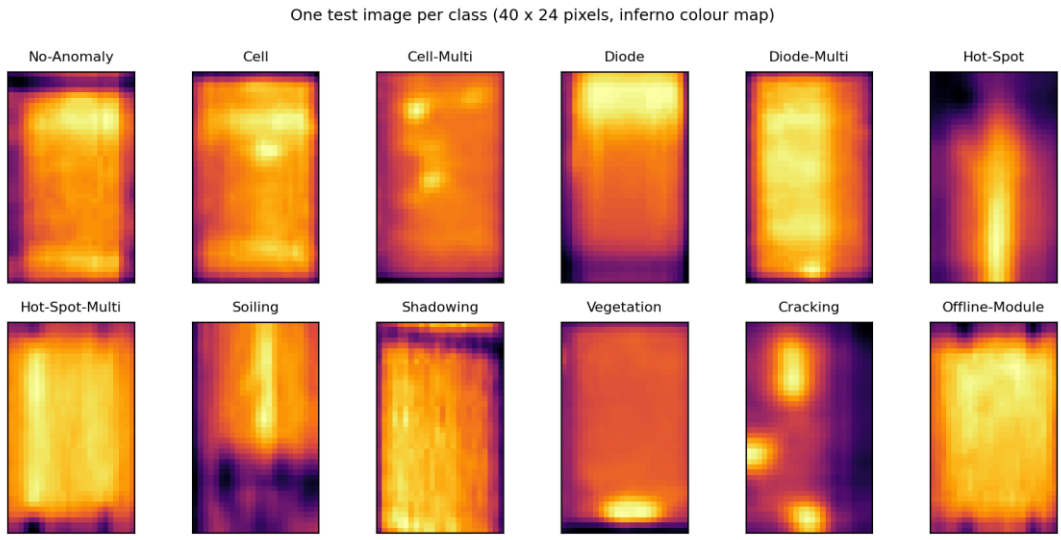



**Fig. 3. One test image per class, shown with the inferno colour map.**

## 5.2.2 Split and Normalisation

The images are split once into 70% training (14,000), 15% validation (3,000) and 15% test (3,000) images, stratified by class with a fixed random seed, so that every model sees exactly the same split (Table 3.1). Pixel values are scaled to [0, 1] and normalised with statistics computed on the training split only, which prevents information from the validation and test images from entering the pipeline:

$$x' = (x - \mu_{train}) / \sigma_{train}, \quad \mu_{train} = 0.618, \quad \sigma_{train} = 0.156$$

## 5.2.3 Augmentation

During training only random horizontal and vertical flips (probability 0.5 each) are applied. A flip does not change the class of a module, because the faults are defined by which cells or diodes are hot and not by orientation. Brightness, contrast and noise jitter were implemented and tested but removed, because in thermal images brightness is temperature, and perturbing it lowered accuracy by about ten points (Section 6.3.1). Quarter-turn rotations are not used, because they would change the 40 x 24 shape and the convolutional pipeline assumes a fixed size.

## 5.2.4 Handling Class Imbalance

Class imbalance is handled at three levels: structurally by the early exit of Stage 1, in the loss by class weights, and in the evaluation by macro-averaged metrics. For a network with K

classes trained on N samples, class r with n_r samples receives the weight

$$w_r = N / (K \times n_r)$$

so that every class contributes equally to the loss in expectation. For example, in the flat 12-class setting the healthy class receives a weight of 0.17 and Diode-Multi a weight of 9.5.

## 5.3 Building Blocks and Their Mathematics

All networks are assembled from a small set of layers. This section states the operation of each layer, so that the architecture tables and parameter counts in the following sections can be checked by hand.

### 5.3.1 Convolution

A convolutional layer with C_in input channels, C_out output channels and a k x k kernel computes, for every output channel c and position (i, j),

$$y[c, i, j] = b[c] + \text{sum over } (m, u, v) \text{ of } W[c, m, u, v] * x[m, i + u, j + v]$$

with padding that preserves the spatial size (k = 3, padding 1). The layer has k^2 x C_in x C_out weights and C_out biases (the bias is omitted when a BatchNorm layer follows).

### 5.3.2 Batch Normalisation

BatchNorm normalises each channel over the mini-batch and then rescales it with two learned parameters per channel:

$$x_hat = (x - mean_B) / sqrt(var_B + eps), \quad y = gamma * x_hat + beta$$

It stabilises and speeds up training and reduces sensitivity to weight initialisation, which matters for small networks trained on imbalanced data. At test time running averages of the mean and variance are used.

### 5.3.3 ReLU and Max Pooling

The rectified linear unit f(x) = max(0, x) introduces the non-linearity that lets the network learn complex decision boundaries. Max pooling with a 2 x 2 window and stride 2 keeps the maximum of each window, halving height and width, reducing computation and giving tolerance to small shifts in where the heat pattern sits in the image.

### 5.3.4 Global Average Pooling and Classification Head

After the last block each of the C feature maps of size H x W is summarised by its mean, z_c = (1 / HW) sum of x_c(i, j). Dropout with probability p randomly zeroes activations during training as a regulariser, and a fully connected layer maps the C values to K class scores s_k. The class probabilities are given by the softmax function:

$$p_k = exp(s_k) / sum\ over\ j\ of\ exp(s_j)$$

### 5.3.5 Loss Function

Each network minimises the class-weighted cross-entropy over a mini-batch of B samples with labels y_i:

$$L = - (1 / B) * sum\ over\ i\ of\ w[y_i] * log\ p[i, y_i]$$

### 5.3.6 Counting Parameters

A block of two convolutions followed by BatchNorm has k^2 x C_in x C_mid + k^2 x C_mid x C_out weights and 2 x (C_mid + C_out) BatchNorm parameters. The tables in the following sections were generated by running the implemented models, and the totals match this formula.

## 5.4 Stage 1: CNN-1, Binary Triage

CNN-1 answers whether a module is healthy or defective. In the from-scratch variant it has three blocks of two 3 x 3 convolutions, each followed by BatchNorm and ReLU, then 2 x 2 max pooling, with 64, 128 and 128 channels, followed by global average pooling, dropout (0.2) and a fully connected layer with two outputs, 555,074 parameters in total. The 80 x 48 upsampled input is reduced to 64 x 40 x 24, 128 x 20 x 12 and 128 x 10 x 6 feature maps. A prediction of No-Anomaly ends the pipeline (the early exit); a defect continues to Stage 2. Table 5.1 lists every layer.

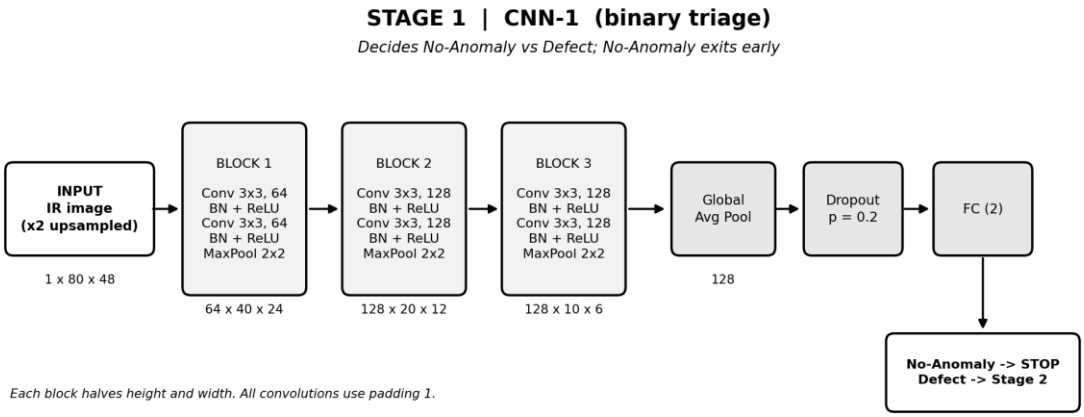



**Fig. 4. Stage 1: CNN-1, binary triage (layers and feature-map sizes).**

**Table 5.1: Layer-by-layer structure of CNN-1 (from-scratch variant)**

| Layer | Output shape | Parameters |
|---|---|---|
| Input | 1 x 80 x 48 | 0 |
| Conv2d(64) | 64 x 80 x 48 | 576 |
| BatchNorm2d | 64 x 80 x 48 | 128 |
| ReLU | 64 x 80 x 48 | 0 |
| Conv2d(64) | 64 x 80 x 48 | 36,864 |
| BatchNorm2d | 64 x 80 x 48 | 128 |
| ReLU | 64 x 80 x 48 | 0 |
| MaxPool2d | 64 x 40 x 24 | 0 |
| Conv2d(128) | 128 x 40 x 24 | 73,728 |
| BatchNorm2d | 128 x 40 x 24 | 256 |
| ReLU | 128 x 40 x 24 | 0 |
| Conv2d(128) | 128 x 40 x 24 | 147,456 |
| BatchNorm2d | 128 x 40 x 24 | 256 |
| ReLU | 128 x 40 x 24 | 0 |
| MaxPool2d | 128 x 20 x 12 | 0 |
| Conv2d(128) | 128 x 20 x 12 | 147,456 |
| BatchNorm2d | 128 x 20 x 12 | 256 |
| ReLU | 128 x 20 x 12 | 0 |
| Conv2d(128) | 128 x 20 x 12 | 147,456 |
| BatchNorm2d | 128 x 20 x 12 | 256 |
| ReLU | 128 x 20 x 12 | 0 |
| MaxPool2d | 128 x 10 x 6 | 0 |
| GlobalAvgPool | 128 | 0 |
| FC(2) | 2 | 258 |
| **Total** | | **555,074** |

## 5.5 Stage 2: CNN-2, Fault-Family Classifier

CNN-2 has the same three-block design with 128, 256 and 128 channels, dropout 0.4 and three outputs, 1,478,147 parameters in total. It is trained only on defective images labelled by family. Its highest-scoring output selects which specialist runs next. Faults of one family should look

alike, so the decision should be easier than choosing among eleven faults directly. Section 6.4 shows how well this holds.

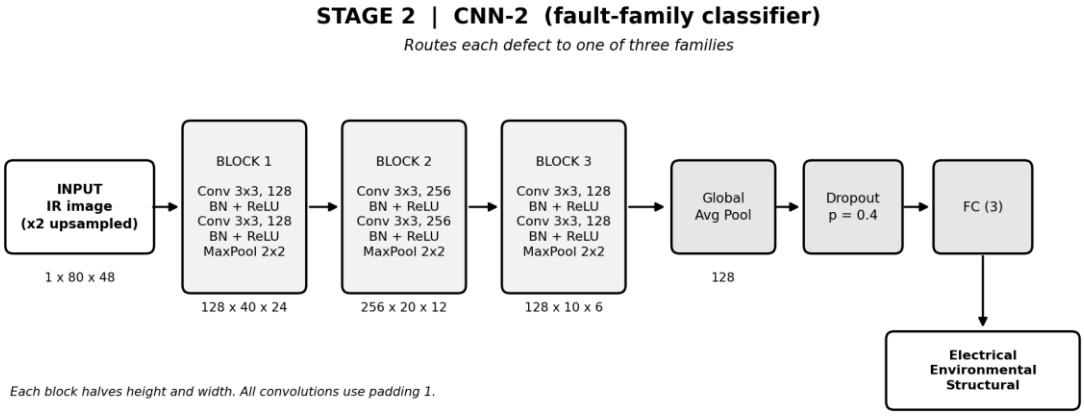



**Fig. 5. Stage 2: CNN-2, fault-family classifier.**

**Table 5.2: Layer-by-layer structure of CNN-2 (from-scratch variant)**

| Layer | Output shape | Parameters |
|---|---|---|
| Input | 1 x 80 x 48 | 0 |
| Conv2d(128) | 128 x 80 x 48 | 1,152 |
| BatchNorm2d | 128 x 80 x 48 | 256 |
| ReLU | 128 x 80 x 48 | 0 |
| Conv2d(128) | 128 x 80 x 48 | 147,456 |
| BatchNorm2d | 128 x 80 x 48 | 256 |
| ReLU | 128 x 80 x 48 | 0 |
| MaxPool2d | 128 x 40 x 24 | 0 |
| Conv2d(256) | 256 x 40 x 24 | 294,912 |
| BatchNorm2d | 256 x 40 x 24 | 512 |
| ReLU | 256 x 40 x 24 | 0 |
| Conv2d(256) | 256 x 40 x 24 | 589,824 |
| BatchNorm2d | 256 x 40 x 24 | 512 |
| ReLU | 256 x 40 x 24 | 0 |
| MaxPool2d | 256 x 20 x 12 | 0 |
| Conv2d(128) | 128 x 20 x 12 | 294,912 |
| BatchNorm2d | 128 x 20 x 12 | 256 |
| ReLU | 128 x 20 x 12 | 0 |
| Conv2d(128) | 128 x 20 x 12 | 147,456 |
| BatchNorm2d | 128 x 20 x 12 | 256 |
| ReLU | 128 x 20 x 12 | 0 |
| MaxPool2d | 128 x 10 x 6 | 0 |
| GlobalAvgPool | 128 | 0 |
| FC(3) | 3 | 387 |
| **Total** | | **1,478,147** |

## 5.6 Stage 3: Fine-Grained Specialists

Three specialists share the 128-256-128 design and differ only in their output layer: CNN-3a

(Electrical, six classes), CNN-3b (Environmental, three classes) and CNN-3c (Structural, two classes). Each is trained only on the defective images of its own family, so it can spend all its capacity on the distinctions inside that family. The final label is the family's class offset plus the specialist's prediction. Table 5.3 gives the parameter counts of all five networks in both variants.

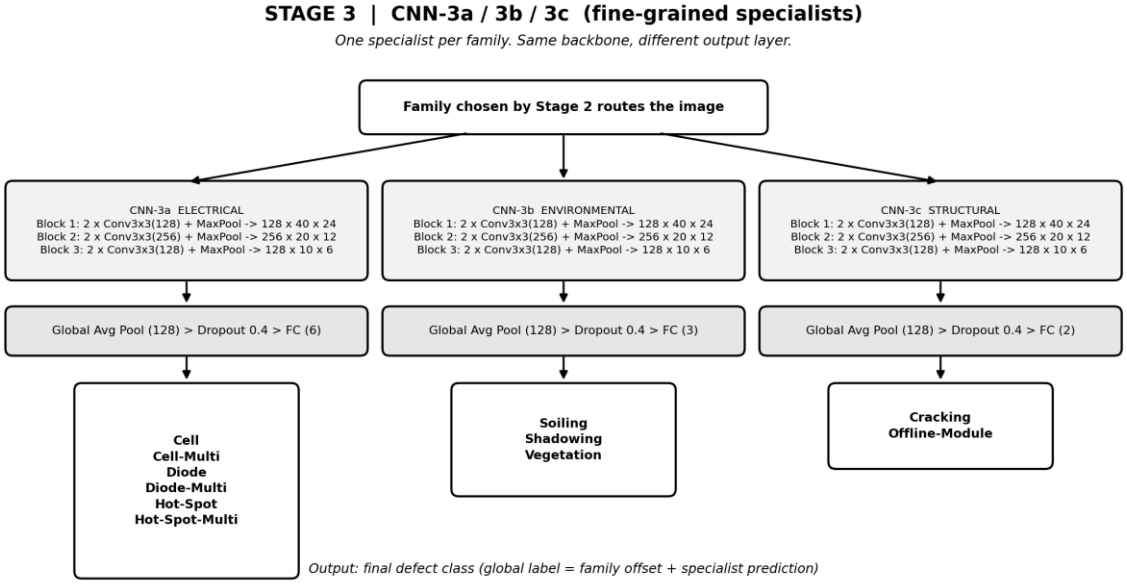



**Fig. 6. Stage 3: the three specialists and their classes.**

**Table 5.3: Parameter counts of the five stage networks**

| Network | Output classes | From scratch | Pretrained EfficientNet-B0 |
|---|---|---|---|
| CNN-1 (triage) | 2 | 555,074 | 4,010,110 |
| CNN-2 (family) | 3 | 1,478,147 | 4,011,391 |
| CNN-3a (Electrical) | 6 | 1,478,534 | 4,015,234 |
| CNN-3b (Environmental) | 3 | 1,478,147 | 4,011,391 |
| CNN-3c (Structural) | 2 | 1,478,018 | 4,010,110 |
| **Cascade total** | | **6,467,920** | **20,058,236** |

**Table 5.4: Architecture summary of the five networks (from-scratch variant)**

| Network | Stage | Input | Layers | Output |
|---|---|---|---|---|
| CNN-1 | 1: triage | IR image, 80 x 48 | 3 blocks 2 x (Conv3x3, BN, ReLU) + MaxPool, 64-128-128; GAP, Dropout 0.2, FC | 2 classes |
| CNN-2 | 2: family | Defective image | Same, 128-256-128; Dropout 0.4, FC | 3 families |
| CNN-3a | 3: Electrical | Routed image | Same as CNN-2, FC with 6 outputs | 6 classes |
| CNN-3b | 3: Environmental | Routed image | Same as CNN-2, FC with 3 outputs | 3 classes |
| CNN-3c | 3: Structural | Routed image | Same as CNN-2, FC with 2 outputs | 2 classes |

## 5.7 Pretrained Variant

In the pretrained variant each of the five networks is an ImageNet-pretrained EfficientNet-B0 [11] whose classifier is replaced by dropout and a fully connected layer with the stage's number of outputs. EfficientNet-B0 is built from mobile inverted bottleneck (MBConv) blocks: each block expands the channels with a 1 x 1 convolution, applies a depthwise convolution, re-weights channels with a squeeze-and-excitation module and projects back with another 1 x 1 convolution, adding a residual connection when the shape allows. The family of networks was designed by compound scaling of depth, width and resolution, which gives B0 about 4 million parameters in the form used here.

The grayscale image is resized to 160 x 96 with bilinear interpolation and repeated to three channels inside the model, so the rest of the pipeline is unchanged and the pretrained first layer can be used as is. The five networks hold 20.1 million parameters in total, against 6.5 million for the from-scratch cascade. All weights are fine-tuned, with a learning rate of 3e-4 and mixed-precision arithmetic.

## 5.8 Training Procedure

Each network is trained independently on the data matching its task (ground-truth routing). Table 5.5 shows the training and validation sizes of the five tasks. Training uses the Adam optimiser with weight decay 1e-4, batch size 64 and the class-weighted loss of Section 5.3.5. The learning rate is 5e-4 for the from-scratch networks and 3e-4 for the pretrained ones. The checkpoint with the best validation macro-F1 is kept, the learning rate is halved when that metric stalls, and training stops after 25 epochs (6 for the pretrained networks) without improvement.

**Table 5.5: Training data of each stage network**

| Network | Task | Training images | Validation images | Classes |
|---|---|---|---|---|
| CNN-1 | Healthy vs defect | 14,000 | 3,000 | 2 |
| CNN-2 | Family of a defect | 7,000 | 1,500 | 3 |
| CNN-3a | Electrical subtype | 3,734 | 801 | 6 |
| CNN-3b | Environmental subtype | 2,029 | 434 | 3 |
| CNN-3c | Structural subtype | 1,237 | 265 | 2 |

**Algorithm 5.1: Training of one stage network**

```
input : training set D_train, validation set D_val, network f, class weights w
output: best checkpoint f*
best <- -1;  wait <- 0
for epoch = 1 .. E_max:
```

```
    for each mini-batch (x, y) of D_train:
        x <- normalise(random_flip(x))
        loss <- weighted_cross_entropy(f(x), y, w)
        update the parameters of f with Adam on loss
    F1_val <- macro_F1(f on D_val)
    if F1_val > best:  best <- F1_val;  save checkpoint f*;  wait <- 0
    else:              wait <- wait + 1;  if wait >= patience: stop
return f*
```

# 5.9 Cascade Inference and Cost Analysis

**Algorithm 5.2: Inference of Cascaded-SolarNet on one image**

```
input : image x
output: class label y
x <- normalise(x)
if CNN-1(x) = No-Anomaly:  return No-Anomaly          # early exit
family <- argmax CNN-2(x)
k      <- argmax CNN-3[family](x)
return class_offset[family] + k
```

Let c1, c2 and c3 be the costs of the three kinds of network. A healthy image costs c1 and a defective one c1 + c2 + c3, so the expected cost per image is c1 + P(defect)(c2 + c3), where P(defect) is the fraction of images that pass Stage 1. On the test split 50.8% of the images leave at Stage 1. With similar-sized networks in all stages the cascade therefore costs about 1.5 network evaluations per image on average, instead of one for a flat model, and it only saves time when the first stage is much cheaper than the others. In the from-scratch variant CNN-1 has about 40% of the parameters of the others, but in the pretrained variant all five networks are the same size, which is why the early exit gave no efficiency advantage in our measurements.

# 5.10 Evaluation Tools and Demonstrations

The evaluation script runs the whole cascade end to end on the test split and reports accuracy, macro-F1, Cohen's kappa, per-class scores, the confusion matrix and the error attribution per stage (missed defects and false alarms at Stage 1, wrong family at Stage 2, wrong subtype at Stage 3). Further scripts reproduce the baselines, run 10-fold cross-validation, search for the data-driven family grouping and generate the figures and reports. Two demonstrations show the cascade at work: a command-line tool that draws each stage's confidence bars for chosen test images (Figure 7), and an interactive browser page in which a class can be selected and the three stages reveal their decisions one after another. Appendix C explains how to run them.

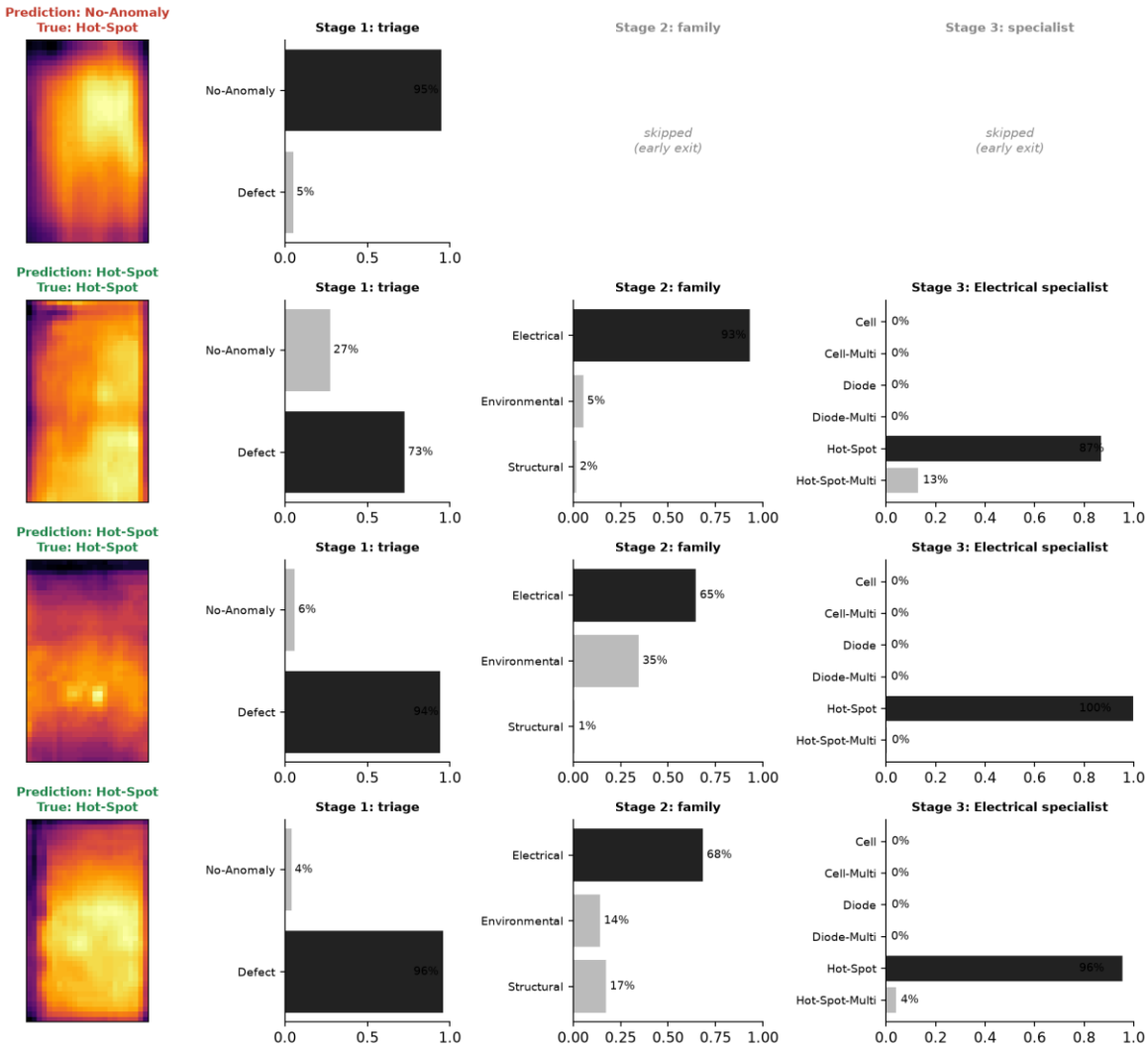



**Fig. 7. Output of the live demonstration: each stage's decision and confidence for Hot-Spot test images.**

## 5.11 Testing and Reproducibility

A suite of 15 automated tests runs on synthetic data and needs no dataset download. It verifies the class hierarchy, the data split, the augmentation, the shapes of all networks and the routing of the cascade. Table 5.6 lists the tests. Reproducibility is supported by a fixed seed, a saved split file, configuration files for every experiment and result files for every reported number.

**Table 5.6: Automated tests**

| Test | What it verifies |
|---|---|
| Twelve classes, healthy first | The taxonomy has 12 classes and the healthy class has global index 0 |
| Node labels cover every class once | Each class belongs to exactly one family and receives a consistent local label |

| Test | What it verifies |
|---|---|
| Global index matches family offsets | Family offset plus local index reproduces the global class index |
| Duplicate class rejected | A taxonomy listing a class twice raises an error |
| Class weights formula | Weights equal N / (K x n) and sum correctly |
| Stratified split | Train, validation and test ids are disjoint, complete and reproducible for a fixed seed |
| Node output shape (5 nodes) | Each stage network outputs one score per class for a 40 x 24 input |
| Augmentation shape and range | Flips and jitter keep the tensor shape and values within [0, 1] |
| Cascade routing and early exit | Healthy images exit at Stage 1 and defects reach the correct specialist and offset |
| Pretrained stage output shape | The EfficientNet-B0 stage outputs the right number of classes |

## 5.12 Chapter Summary

The system consists of five small stage networks in two variants, a leak-free data pipeline, a class-weighted training procedure, an evaluation module that attributes every error to a stage and an automated test suite. The next chapter evaluates it.

# CHAPTER 6

# RESULTS AND ANALYSIS

## 6.1 Experimental Setup

### 6.1.1 Dataset and Protocol

All numbers are measured on the 3,000-image held-out test split of Table 3.1 and are single runs per configuration (seed 42). The test images are not used for training, early stopping, normalisation or model selection. Choices such as the learning rate were made on the validation split.

### 6.1.2 Implementation and Training Configuration

**Table 6.1: Training configuration of the evaluated models**

| Group | Parameter | Value |
|---|---|---|
| Data | Split | Stratified 70/15/15 (14,000 / 3,000 / 3,000), seed 42 |
| Data | Normalisation | Mean and standard deviation of the training split |
| Data | Augmentation | Random horizontal and vertical flips (p = 0.5) |
| From scratch | Input size | 80 x 48 (2x bilinear upsampling) |
| From scratch | Blocks and channels | 3 blocks of 2 x (Conv3x3, BN, ReLU) + MaxPool; 64-128-128 (Stage 1), 128-256-128 (others) |
| From scratch | Dropout | 0.2 (Stage 1), 0.4 (Stage 2 and 3) |
| From scratch | Learning rate, patience | 5e-4; 25 epochs (maximum 150) |
| Pretrained | Backbone | EfficientNet-B0, ImageNet weights, new head |
| Pretrained | Input size | 160 x 96, channel repeated to 3 |
| Pretrained | Dropout | 0.2 (Stage 1), 0.3 (others) |
| Pretrained | Learning rate, patience | 3e-4; 6 epochs (maximum 25), mixed precision |
| Common | Optimiser | Adam, weight decay 1e-4, batch size 64 |
| Common | Loss | Class-weighted cross-entropy, w = N / (K x n) |
| Common | Model selection | Early stopping on validation macro-F1 |
| Common | Hardware, software | NVIDIA RTX 5060 Laptop GPU; PyTorch 2.11 |

### 6.1.3 Evaluation Metrics

With TP, FP and FN the true positives, false positives and false negatives of a class, precision is TP / (TP + FP), recall is TP / (TP + FN) and the F1 score is their harmonic mean:

$$F1 = 2 \times precision \times recall / (precision + recall), \quad macro\text{-}F1 = (1 / K) \times sum\ over\ classes\ of\ F1_r$$

Macro-F1 weights every class equally and therefore exposes poor performance on rare faults, which accuracy hides under class imbalance. Cohen's kappa measures agreement beyond

chance:

$$kappa = (p_o - p_e) / (1 - p_e)$$

where p_o is the observed accuracy and p_e the accuracy expected by chance from the class frequencies. The Stage 1 binary accuracy measures the healthy-versus-defect decision, and per-stage error counts attribute each mistake to the first stage that went wrong.

### 6.1.4 Configurations

**Table 6.2: Definition of the evaluated configurations**

| ID | Variant | Modification |
|---|---|---|
| C1 | Default | Flips plus brightness (20%), contrast (15%) and noise jitter; 40 x 24 input; 60 epochs |
| C2 | No photometric jitter | C1 with flips only; longer schedule (100 epochs) |
| C3 | Improved | C2 with 80 x 48 input and two convolutions per block; 150 epochs |
| C4 | Improved2 | C3 with balanced sampling and label smoothing instead of loss weights |
| C5 | Improved3 | C3 with larger Stage 1 and Stage 2 networks |
| C6 | Data-driven families | C5 with the data-driven family grouping of Section 4.3 |
| C7 | Pretrained | EfficientNet-B0 (ImageNet) fine-tuned at every stage |

## 6.2 Overall Performance

### 6.2.1 Test Results

The best result is the pretrained cascade (C7) with 83.2% accuracy, macro-F1 0.700, Cohen's kappa 0.764 and 95.8% on the Stage 1 healthy-versus-defect decision. The best from-scratch cascade reaches 79.8% accuracy. Table 6.3 lists all configurations and baselines, and Figure 8 plots them.

**Table 6.3: Test results by configuration**

| Configuration | Model | Accuracy | Macro-F1 | Kappa | Stage 1 binary |
|---|---|---|---|---|---|
| C1 Default (jitter on) | Cascade | 61.7% | 0.368 | 0.461 | 81.0% |
| | Flat CNN | 47.6% | 0.379 | 0.357 | n/a |
| C2 No photometric jitter | Cascade | 71.1% | 0.509 | 0.583 | 86.0% |
| | Flat CNN | 68.5% | 0.571 | 0.586 | n/a |
| C3 Improved (80x48, deeper) | Cascade | 76.0% | 0.582 | 0.656 | 89.5% |
| | Flat CNN | 76.8% | 0.653 | 0.689 | n/a |
| C4 Improved2 (balanced sampling) | Cascade | 75.5% | 0.576 | 0.648 | 89.1% |
| C5 Improved3 (bigger Stage 1/2) | Cascade | 79.6% | 0.638 | 0.714 | 93.8% |
| C6 Data-driven families | Cascade | 79.8% | 0.646 | 0.717 | 94.1% |
| **C7 Pretrained EfficientNet-B0** | **Cascade** | **83.2%** | **0.700** | **0.764** | **95.8%** |
| | Flat CNN | 82.6% | 0.708 | 0.761 | n/a |
| Raw-pixel Random Forest | RF | 61.6% | 0.337 | 0.324 | n/a |
| EfficientNet-B0 features + SVM | SVM | 78.7% | 0.640 | 0.689 | 90.4% |

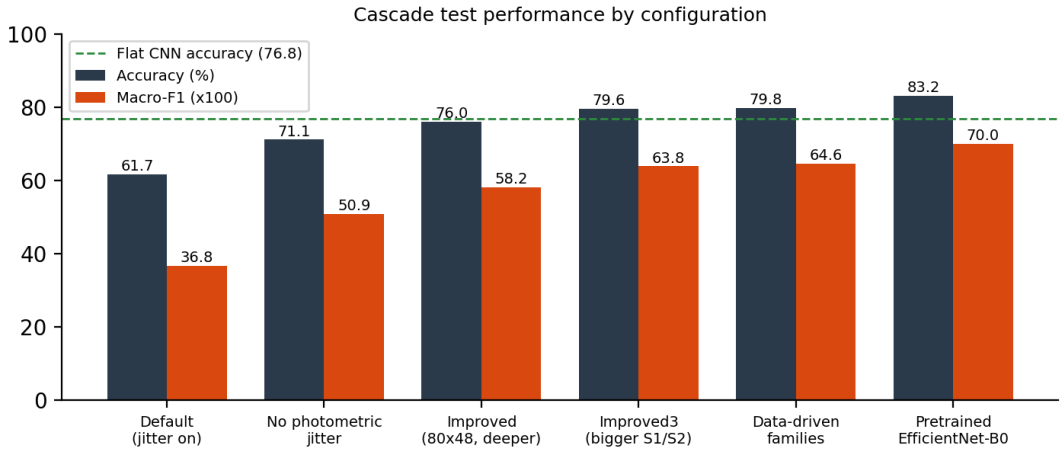



**Fig. 8. Cascade test accuracy and macro-F1 by configuration (dashed line: from-scratch flat CNN).**

## 6.2.2 Per-Network Validation Results and Training Behaviour

Figure 9 compares the best validation macro-F1 of each of the five networks in the two variants, and Figure 10 shows the training curves. Pretraining improved every network, most clearly Stage 2 (validation macro-F1 0.717 to 0.796) and the Electrical specialist (0.786 to 0.826). The pretrained networks reach their best validation score within 10 to 20 epochs, while the from-scratch networks need 50 to 140 epochs, and the validation curves of the from-scratch family and electrical networks are noticeably noisier.

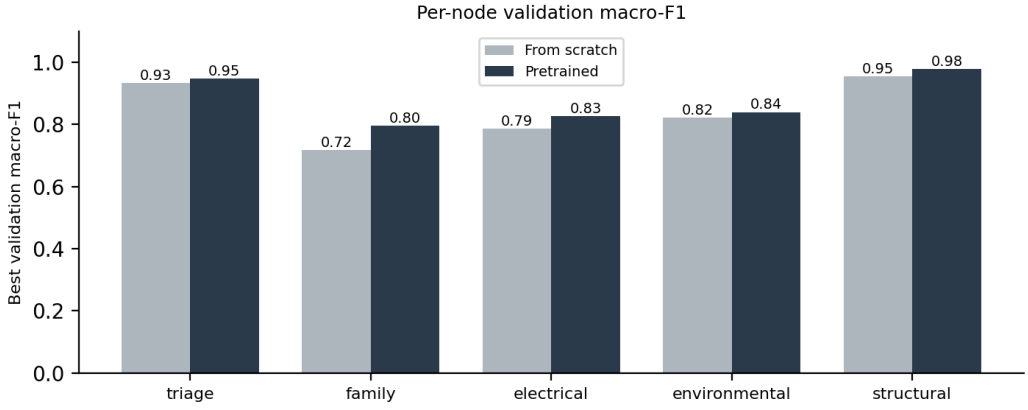



**Fig. 9. Best validation macro-F1 of each stage network, from scratch and pretrained.**

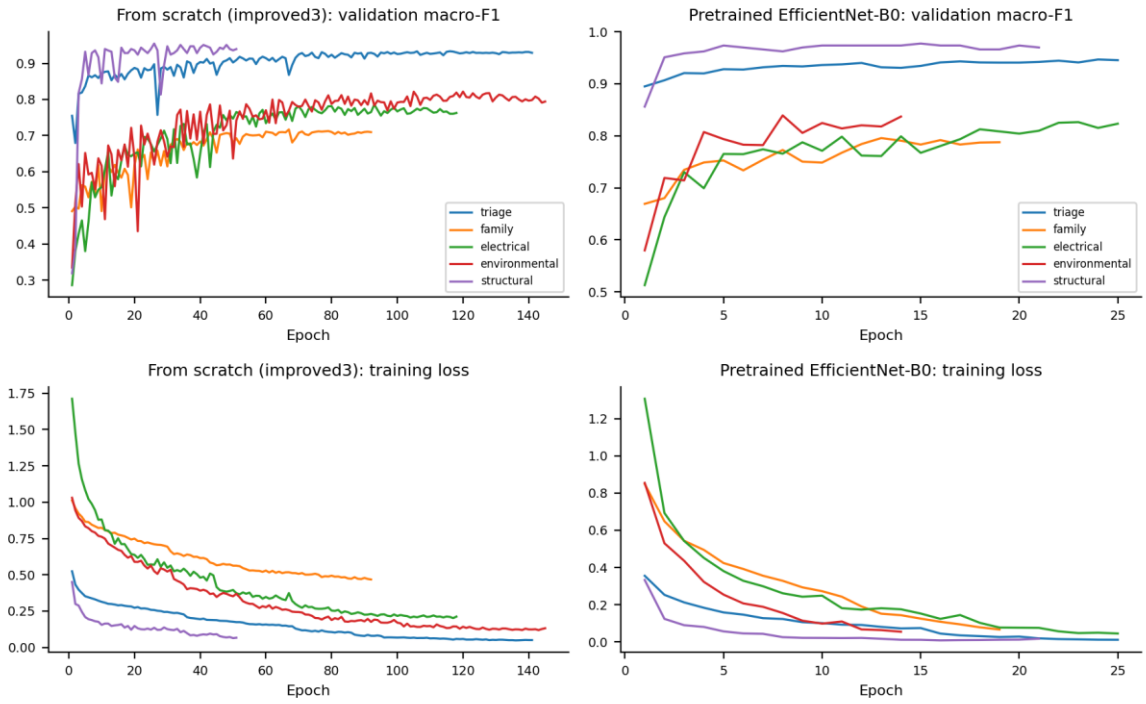



**Fig. 10. Validation macro-F1 and training loss of the five stage networks.**

## 6.3 Ablation Study

The ablations change one design choice at a time, in the order in which the project progressed (Table 6.2). Each subsection reports what changed and what was measured.

### 6.3.1 Photometric Augmentation

**Table 6.4: Effect of photometric augmentation (cascade, 40 x 24 input)**

| Configuration | Accuracy | Macro-F1 | Kappa | Stage 1 binary |
|---|---|---|---|---|
| C1 with brightness, contrast and noise jitter | 61.7% | 0.368 | 0.461 | 81.0% |
| C2 flips only | 71.1% | 0.509 | 0.583 | 86.0% |

Removing the jitter raised accuracy by about ten points and cut Stage 1 false alarms from 250 to 118. In an infrared image the absolute brightness of a region is its temperature, which is exactly what separates a hot spot from a healthy module, so scaling or shifting brightness changes the evidence and not only the appearance. This was the largest single gain of the project and the main lesson about thermal data.

### 6.3.2 Input Size and Network Capacity

**Table 6.5: Effect of input size and capacity (cascade, flips only)**

| Configuration | Accuracy | Macro-F1 | Kappa | Stage 1 binary |
|---|---|---|---|---|
| C2 40 x 24 input | 71.1% | 0.509 | 0.583 | 86.0% |
| C3 80 x 48 input, two convolutions per block | 76.0% | 0.582 | 0.656 | 89.5% |
| C5 plus larger Stage 1 and Stage 2 networks | 79.6% | 0.638 | 0.714 | 93.8% |

Upsampling the image to 80 x 48 and adding a second convolution per block gave 76.0% accuracy, and enlarging the two front stages gave 79.6%. Stage 1 missed defects fell from 233 to 108. These changes target the early stages because their errors cannot be corrected later.

### 6.3.3 Balanced Sampling and Label Smoothing

Replacing the class-weighted loss by sampling with weight proportional to 1/sqrt(class count) and adding label smoothing of 0.05 (C4) gave 75.5% accuracy and macro-F1 0.576, slightly below the weighted loss (76.0%, 0.582). The class-weighted loss was therefore kept.

### 6.3.4 Inference-Time Variants

**Table 6.6: Inference-time variants on the improved cascade (no retraining)**

| Variant | Accuracy | Macro-F1 | Kappa |
|---|---|---|---|
| Hard routing (baseline) | 76.0% | 0.582 | 0.656 |
| Flip test-time augmentation | 76.4% | 0.582 | 0.660 |
| Stage 1 threshold tuned on validation data | 76.0% | 0.582 | 0.656 |
| Soft routing (multiplied probabilities) | 75.9% | 0.575 | 0.648 |
| Soft routing with test-time augmentation | 76.3% | 0.579 | 0.653 |

None of the variants helped meaningfully. The validation-optimal Stage 1 threshold stayed at 0.5, so tuning it changed nothing. Soft routing, which multiplies the probabilities of the stages so that a doubtful family decision can be outvoted, did not recover the Stage 2 errors, which suggests that those errors are confident mistakes and not close calls. Test-time augmentation added 0.3 points at four times the cost.

### 6.3.5 Fault-Family Grouping

Table 6.7 compares the semantic grouping with the data-driven grouping of Section 4.3. The data-driven grouping reduced wrong-family errors at Stage 2 from 300 to 116, but moved the difficulty into the fine-grained stage, where its largest family now contains the visually similar classes Cell, Cell-Multi, Soiling, Vegetation and Cracking. The net effect on accuracy was

small.

**Table 6.7: Semantic versus data-driven family grouping (from-scratch cascade)**

| Grouping | Stage 2 errors | Stage 3 errors | Total errors | Accuracy | Macro-F1 |
|---|---|---|---|---|---|
| Semantic (Electrical, Environmental, Structural) | 300 | 126 | 613 | 79.6% | 0.638 |
| Data-driven (groups A, B, C) | 116 | 313 | 605 | 79.8% | 0.646 |

### 6.3.6 ImageNet Pretraining

Fine-tuning ImageNet-pretrained EfficientNet-B0 at every stage raised the cascade from 79.8% to 83.2% accuracy and macro-F1 from 0.646 to 0.700, and training took minutes instead of an hour. The pretrained flat CNN improved even more, from 76.8% to 82.6%, so that the two designs end up tied. Pretraining helps on these small thermal images because the early filters (edges, blobs, gradients) transfer, which refutes the expectation in the project proposal that pretrained backbones would give little benefit.

### 6.3.7 Summary of the Ablations

In order of effect: removing photometric jitter (+9.4 points), pretraining (+3.4 and +5.8 for the flat model), larger front stages and input (+3.6 and +5.0 on the way), while sampling changes, inference variants and test-time augmentation gave nothing. The changes that helped were those that preserved or exposed the thermal signal or brought in better features. The changes that did not help tried to repair Stage 2 after the fact.

## 6.4 Error Analysis by Stage

Table 6.8 attributes each test error to the first stage that went wrong.

**Table 6.8: Test errors per stage (out of 3,000 images)**

| Configuration | Stage 1 missed defect | Stage 1 false alarm | Stage 2 wrong family | Stage 3 wrong subtype | Total |
|---|---|---|---|---|---|
| C3 Improved | 233 | 83 | 294 | 109 | 719 |
| C5 Improved3 | 108 | 79 | 300 | 126 | 613 |
| C6 Data-driven | 110 | 66 | 116 | 313 | 605 |
| **C7 Pretrained** | **75** | **52** | **272** | **106** | **505** |

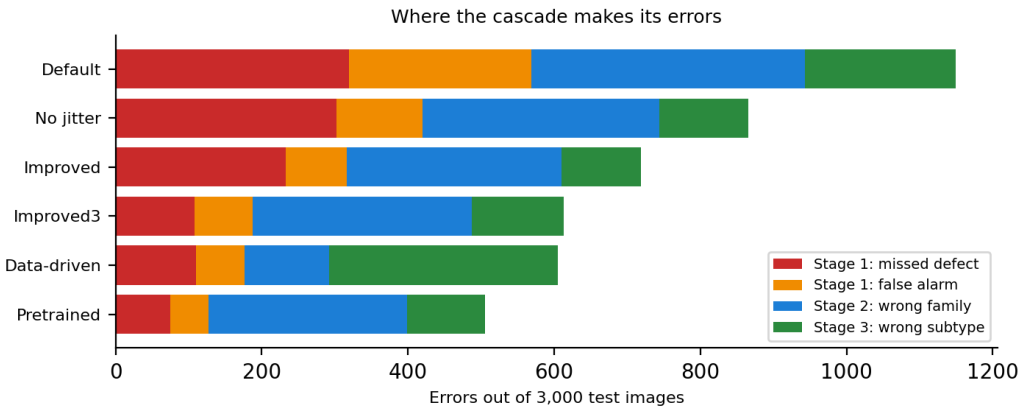



**Fig. 11. Where the cascade makes its errors, by configuration.**

Pretraining reduced errors at every stage compared with C5 (total 613 to 505), but Stage 2, the family decision, remains the largest source (272 of 505 errors), and an error there cannot be corrected by Stage 3. About 50.8% of the test images leave the cascade at Stage 1. Stage 1 itself is accurate: only 75 defects were missed and 52 healthy modules flagged, a binary accuracy of 95.8%. The missed defects matter most in practice, because a missed hot spot is a safety risk, whereas a false alarm costs only an inspection.

## 6.5 Per-Class Performance and Confusion

### 6.5.1 Per-Class F1

The rare classes are the weak point. They have only 123 to 174 training images, and several look alike. Class weighting, stratified splitting and macro-F1 reduce but do not remove the problem.

**Table 6.9: Per-class F1 on the test set**

| Class | Test images | Cascade, from scratch (C5) | Cascade, pretrained (C7) | Flat CNN, pretrained |
|---|---|---|---|---|
| No-Anomaly | 1500 | 0.94 | 0.96 | 0.94 |
| Cell | 281 | 0.63 | 0.69 | 0.69 |
| Cell-Multi | 193 | 0.47 | 0.52 | 0.49 |
| Diode | 225 | 0.93 | 0.96 | 0.96 |
| Diode-Multi | 26 | 0.82 | 0.87 | 0.91 |
| Hot-Spot | 37 | 0.54 | 0.65 | 0.62 |
| Hot-Spot-Multi | 37 | 0.57 | 0.65 | 0.62 |
| Soiling | 31 | 0.22 | 0.30 | 0.35 |
| Shadowing | 159 | 0.67 | 0.72 | 0.74 |
| Vegetation | 246 | 0.65 | 0.65 | 0.71 |
| Cracking | 141 | 0.65 | 0.70 | 0.75 |
| Offline-Module | 124 | 0.59 | 0.74 | 0.71 |

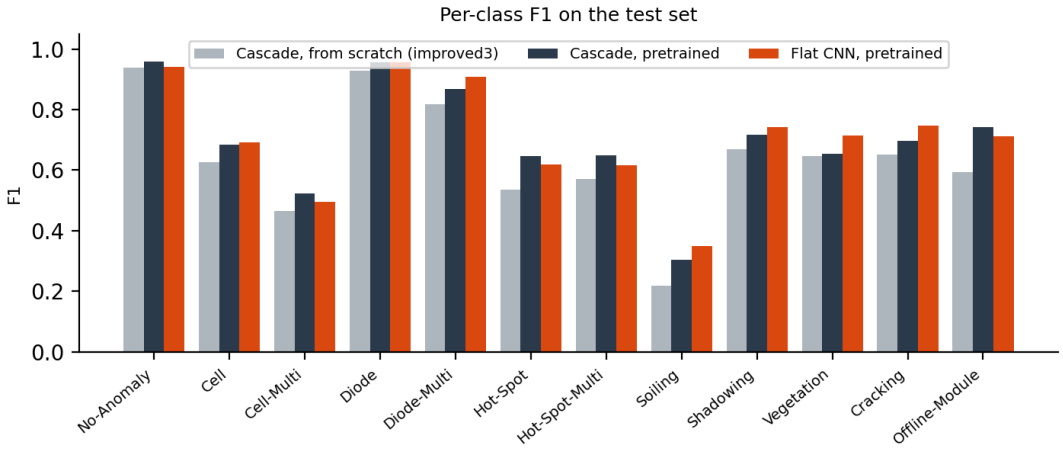



**Fig. 12. Per-class F1 on the test set.**

## 6.5.2 Confusion Matrices

Figures 13 and 14 show the row-normalised confusion matrices of the pretrained cascade and the pretrained flat CNN. Both are diagonal for the healthy class and for Diode, Diode-Multi and Cracking, and both spread the probability mass of Cell, Cell-Multi, Hot-Spot and Soiling over several neighbouring classes. Table 6.10 lists the most frequent confusions of the cascade.

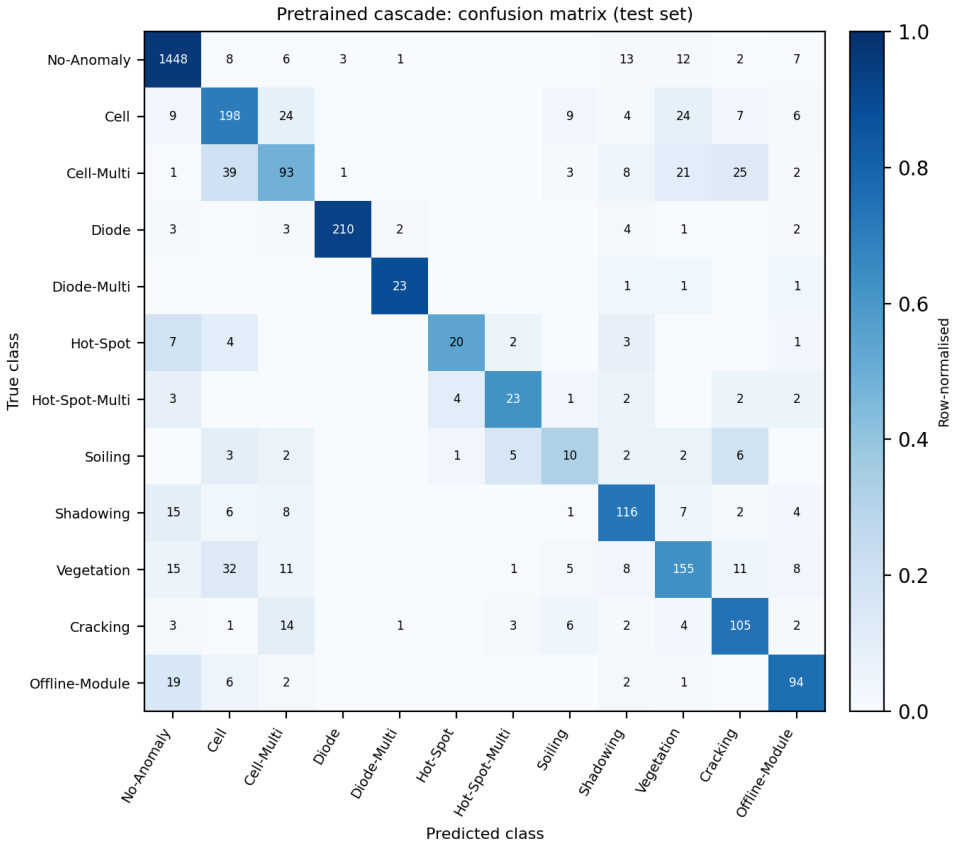



**Fig. 13. Confusion matrix of the pretrained cascade on the test set (row-normalised, counts inside cells).**

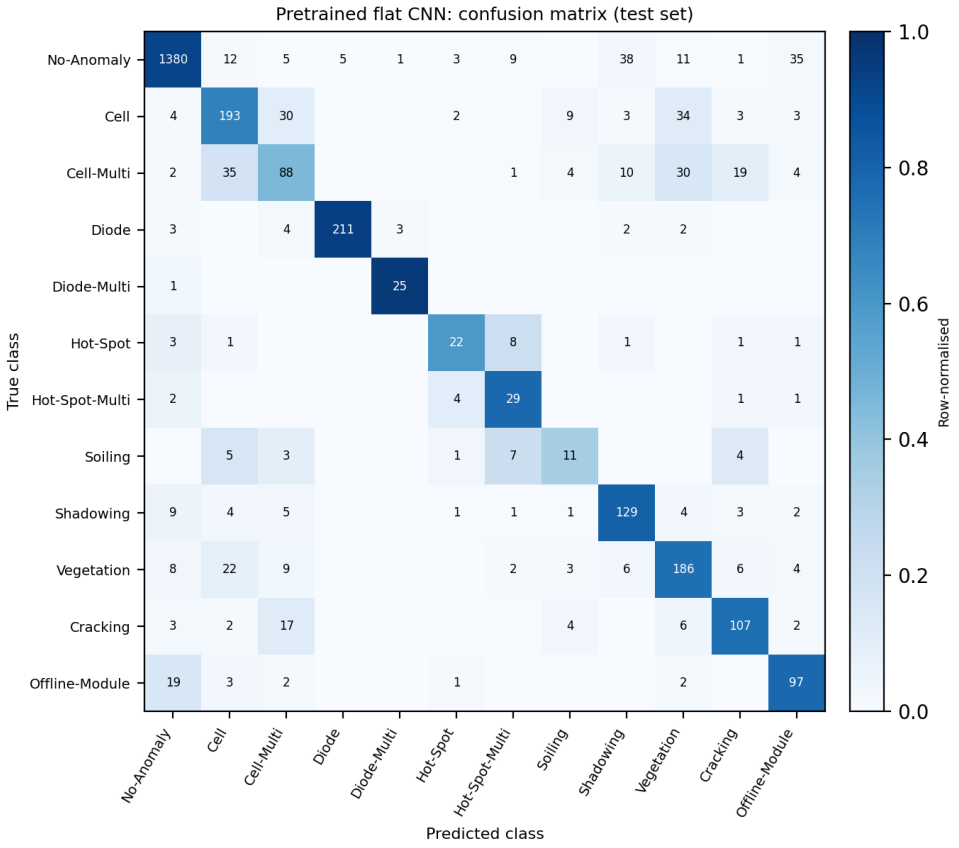



**Fig. 14. Confusion matrix of the pretrained flat CNN on the test set.**

**Table 6.10: Most frequent confusions of the pretrained cascade**

| True class | Predicted as | Images | Share of the true class |
|---|---|---|---|
| Cell-Multi | Cell | 39 | 20% |
| Vegetation | Cell | 32 | 13% |
| Cell-Multi | Cracking | 25 | 13% |
| Cell | Vegetation | 24 | 9% |
| Cell | Cell-Multi | 24 | 9% |
| Cell-Multi | Vegetation | 21 | 11% |

The most frequent confusions are between classes of the same physical phenomenon (a single versus several affected cells or diodes) and between faults and the healthy class, in particular for faint hot spots. These are also the cases where a human inspector would hesitate in a 24 x 40 pixel image.

## 6.6 Comparison with Published Work

The dataset paper reports no model accuracies, and the figures often attributed to it (Random Forest 84%, CNN 92.5%, EfficientNet + SVM 93%) come from later papers that differ in task,

protocol and features. Table 6.11 therefore places published results next to ours but does not rank them.

**Table 6.11: Published results on InfraredSolarModules and results of this project**

| Paper | Method | Task and protocol | Reported | Checked |
|---|---|---|---|---|
| Duranay 2023 [2] | EfficientNet-B0 features (ImageNet), NCA selection, SVM | 12 classes, 10-fold cross-validation | 93.93% accuracy, 89.82% F1 | Full paper read |
| Alves et al. 2021 [3] | Custom CNN with augmentation | Binary detection; 8 selected classes; cross-validation | 92.5% binary; 78.85% for 8 classes | Abstract only |
| Corley et al. 2025 [4] | Random Forest | Anomaly classification, split not stated | 84.0% | Paper text read |
| Barraz 2025; Gopalakrishnan 2024, as cited in [4] | Cascading ensemble; NASNet + LSTM | Not stated | 88%; 85% | As cited |
| Millendorf et al. 2020 [1] | Dataset paper | - | No model results | Full paper read |
| **Ours: pretrained cascade** | **EfficientNet-B0 at each of 5 stages** | **12 classes, fixed 70/15/15 hold-out** | **83.2%, macro-F1 0.700** | **Our results** |
| Ours: from-scratch cascade | Five small CNNs | Same hold-out | 79.8%, macro-F1 0.646 | Our results |
| Ours: Stage 1 only (pretrained) | Binary triage | Healthy vs defect, same hold-out | 95.8% | Our results |
| Ours: flat CNN / Random Forest | EfficientNet-B0 / raw pixels | 12 classes, same hold-out | 82.6% / 61.6% | Our results |
| Ours: Duranay-style baseline | EfficientNet-B0 features + SVM | 12 classes, same hold-out | 78.7%, macro-F1 0.640 | Our results |

- **Binary triage.** Our pretrained Stage 1 reaches 95.8% (94.1% from scratch), above the 92.5% of Alves et al. and below the 97.32% of a multi-scale transfer-learning CNN cited by Duranay. It is a single hold-out run, so we claim to be at or slightly above the 92.5% to 93% range and not a new best.
- **12 classes.** Our best, 83.2%, is below the published 12-class figures (84.0%, 85% to 88% for ensembles, 93.93%). We did not reach the 93% ceiling or the 95% target stated in the proposal. A Duranay-style pipeline run on our split scored only 78.7%, below our pretrained cascade, which suggests that the gap to 93.93% is largely due to the evaluation protocol rather than to a weaker model; this is a suggestion and not a proof.
- **Why the results are not like for like.** Duranay uses 10-fold cross-validation, a network trained end to end on the dataset and a supervised feature selection before the cross-validation, so as described, test images can influence these steps. Our test images never do. A raw-pixel Random Forest scores 61.6% on our split against 84.0% in [4], whose protocol is not stated, so part of the gap remains unexplained.

## 6.7 Data Overlap Analysis

The benchmark has no module or site identifiers, so near-identical crops of the same module can exist. To measure how much this matters, every test image was compared with every training image by the Euclidean distance between their 24 x 40 pixel vectors. Table 6.12 shows the result. A raw-pixel nearest-neighbour classifier with no training already reaches 62.4% accuracy, the same as the raw-pixel Random Forest (61.6%).

**Table 6.12: Similarity between test and training images (raw pixels)**

| Measure | Value |
|---|---|
| Test images with a training image within distance 0.5 | 7.4% |
| Label agreement for those images | 95.9% |
| Test images with a training image within distance 1.0 | 53.5% |
| Label agreement for those images | 78.1% |
| Median distance to the nearest training image | 0.96 |
| Median distance between random image pairs | 4.70 |

About 7% of the test images have a near-identical training image and about half have a close one, so random fold splits, as used in cross-validated studies, would place such near-duplicates on both sides and raise their scores. This explains part of the difference to published numbers but not all of it, since a Random Forest scores far lower on our split than the 84.0% reported elsewhere.

## 6.8 Discussion and Limitations

## 6.8.1 Interpretation of Findings

Three results stand out. Thermal-aware augmentation mattered more than any architectural change. Pretraining gave a large and cheap gain even on 24 x 40 pixel thermal images. And the central hypothesis, that a cascade clearly beats a flat classifier, was only partly supported: the family decision is the weak link, and with equal backbones the two designs tie. The cascade does give something a flat model cannot: it tells where an error came from, and it shows that Stage 1 is a reliable gatekeeper.

## 6.8.2 Limitations

- **Single runs.** Every configuration was run once with seed 42; differences below about 0.5 points are within noise and no confidence intervals were computed. A 10-fold cross-validation of the pretrained cascade was prepared but did not complete, so it is not reported.

- **Near-duplicate images.** The benchmark has no module or site identifiers, so near-identical crops can fall on both sides of any split.
- **Rare classes.** Soiling, Hot-Spot and Diode-Multi remain weak, which keeps macro-F1 well below accuracy.
- **Efficiency.** Latency was not measured reliably, because timings were taken while the GPU was busy, and the pretrained cascade has five times the parameters of the flat model, so the early exit did not give a demonstrated efficiency advantage.
- **Comparison with published work.** Protocols differ, and some sources could only be checked through abstracts or citing papers.

### 6.8.3 Threats to Validity

Internal validity is limited by the single seed and by the fact that hyper-parameters were tuned on one validation split. External validity is limited to one dataset of small crops from one provider; the results may not transfer to higher-resolution images or other sites. Construct validity is limited by label noise, which cannot be assessed because no second annotation exists.

## 6.9 Chapter Summary

The best cascade reaches 83.2% accuracy with ImageNet pretraining and 79.8% from scratch, with Stage 1 at 95.8%. The cascade matches but does not clearly beat a flat classifier, the family stage is its largest error source, and published 12-class results are higher under protocols that are not comparable with a fixed hold-out.

# CHAPTER 7

# CONCLUSION AND FUTURE SCOPE

## 7.1 Conclusion

This project implemented Cascaded-SolarNet, a three-stage cascade of five CNNs for 12-class infrared solar module defect classification, with a reproducible split, tests, architecture diagrams and live demonstrations. After removing harmful photometric augmentation, enlarging the input and strengthening the front stages, accuracy rose from 61.7% to 79.8% from scratch and to 83.2% with ImageNet pretraining (macro-F1 0.37 to 0.70). Healthy-versus-defect triage reaches 95.8%, comparable to the 92.5% binary figure published for this dataset.

The claim that a cascade clearly beats a flat classifier was only partly supported. From scratch the cascade is ahead on accuracy and kappa; with the same pretrained backbone it ties a flat CNN while using five times the parameters. Error propagation is the central trade-off: a wrong decision at Stage 1 or Stage 2 cannot be undone, and the family decision remains the largest error source. Published 12-class results (84% to 93.93%) are higher than ours, but they use cross-validation and pretrained features, and the 93% ceiling and 95% target of the proposal were not reached under a stricter fixed hold-out. A practical lesson is that thermal images need thermal-aware augmentation: flips help, while brightness, contrast and noise jitter hurt.

## 7.2 Future Scope

- **Like-for-like evaluation.** Run several seeds and a leak-free 10-fold cross-validation, and replicate the full NCA pipeline, for a fair comparison with published results.
- **Better family routing.** Train Stage 2 jointly with the specialists or on the cascade's own routing errors, or use soft hand-offs trained end to end.
- **Rare classes.** Apply class-specific augmentation or synthetic oversampling for Soiling, Hot-Spot and Diode-Multi, and deduplicate near-identical crops before splitting.
- **Edge deployment.** Measure real latency, memory and energy on a drone-class device, and test distillation of the pretrained cascade into smaller networks.
- **Additional modalities.** Investigate RGB or contextual information from neighbouring panels, which the crop-level dataset cannot provide.

# REFERENCES


[1] M. Millendorf, E. Obropta, and N. Vadhavkar, "Infrared Solar Module Dataset for Anomaly Detection," ICLR 2020 Workshop on AI for Earth Sciences, 2020.

[2] Z. B. Duranay, "Fault Detection in Solar Energy Systems: A Deep Learning Approach," Electronics, vol. 12, no. 21, p. 4397, 2023, doi: 10.3390/electronics12214397.

[3] R. H. F. Alves, G. A. de Deus Junior, E. G. Marra, and R. P. Lemos, "Automatic fault classification in photovoltaic modules using Convolutional Neural Networks," Renewable Energy, vol. 179, pp. 502-516, 2021.

[4] Corley et al., "Aerial Infrared Health Monitoring of Solar Photovoltaic Farms at Scale," CVPR Workshops (PBVS), 2025, arXiv:2503.02128.

[5] S. P. Pathak and S. A. Patil, "Enhanced Fault Identification in Solar Panels through Binary Cascaded Convolutional Classifiers with Thermal-Visual Augmentation," Int. J. Image, Graphics and Signal Processing, vol. 17, no. 4, 2025.

[6] "PV-faultNet: Optimized Convolutional Neural Network to Detect Defects in PV Production," arXiv:2411.02997, 2024.

[7] "Defect Detection of Photovoltaic Modules Based on Improved VarifocalNet," Scientific Reports, vol. 14, 2024, doi: 10.1038/s41598-024-66234-3.

[8] U. Otamendi, I. Martinez, M. Quartulli, et al., "Segmentation of Cell-Level Anomalies in Electroluminescence Images of Photovoltaic Modules," Solar Energy, vol. 220, pp. 914-926, 2021.

[9] H. Tella, A. Hussein, S. Rehman, et al., "PV Panel Defect Classification Using Deep Learning Ensemble Methods," Case Studies in Thermal Engineering, vol. 66, 2025.

[10] Z. Cai and N. Vasconcelos, "Cascade R-CNN: Delving into High Quality Object Detection," in Proc. CVPR, 2018, pp. 6154-6162.

[11] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. ICML, 2019, arXiv:1905.11946.

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. CVPR, 2016, pp. 770-778.

[13] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning

Library," in Advances in Neural Information Processing Systems 32, 2019.

[14] J. Deng et al., "ImageNet: A Large-Scale Hierarchical Image Database," in Proc. CVPR, 2009.

# APPENDIX A

# CONFIGURATION FILES

This appendix reproduces the two configuration files that define the best from-scratch experiment and the pretrained experiment. Every reported result can be regenerated from the configuration, the saved split file and the seed.

**Listing A.1: configs/improved3.yaml (from-scratch cascade)**

```
# improved3 = improved with larger triage (CNN-1) and family (CNN-2) networks.
# Fine CNNs are copied from runs_improved; only triage and family are retrained.
# Improved: 80x48 upsampling, two convs per block, no photometric jitter, longer
schedule.
data:
  root: data/InfraredSolarModules     # folder containing module_metadata.json
  splits_file: data/splits.json       # written by scripts/make_splits.py
  seed: 42
  val_frac: 0.15
  test_frac: 0.15
  upsample: 2                         # 1 = native 40x24; 2 = 80x48 (optional, see
report 8.5)

augment:
  hflip: 0.5
  vflip: 0.5
  brightness: 0.0
  contrast: 0.0
  noise_std: 0.0

train:
  batch_size: 64
  epochs: 150
  lr: 5.0e-4
  weight_decay: 1.0e-4
  patience: 25                        # early stopping on val macro-F1
  weighted_loss: true                 # w_r = N / (K * n_r)
  num_workers: 2
  out_dir: runs_improved3

# Per-stage CNN widths (poster, "Class Hierarchy" panel).
# Each entry is a Conv3x3-BN-ReLU-MaxPool block; head is GlobalAvgPool + FC.
models:
  triage:     {channels: [64, 128, 128],  dropout: 0.2, convs_per_block: 2}   # CNN-1:
No-Anomaly vs Defect
  family:     {channels: [128, 256, 128], dropout: 0.4, convs_per_block: 2}   # CNN-2:
Electrical / Environmental / Structural
  fine:       {channels: [128, 256, 128], dropout: 0.4, convs_per_block: 2}  # CNN-
3a/b/c: one per family
  flat:       {channels: [128, 256, 128], dropout: 0.4, convs_per_block: 2}  # single-
stage 12-class baseline
```

**Listing A.2: configs/pretrained.yaml (pretrained cascade)**

```
# Pretrained cascade: an ImageNet-pretrained EfficientNet-B0 under every stage, fine-
tuned.
# Inputs are resized inside the model (to `size`), so data.upsample stays 1.
data:
  root: data/InfraredSolarModules     # folder containing module_metadata.json
  splits_file: data/splits.json       # written by scripts/make_splits.py
  seed: 42
```

```
  val_frac: 0.15
  test_frac: 0.15
  upsample: 1

augment:
  hflip: 0.5
  vflip: 0.5
  brightness: 0.0
  contrast: 0.0
  noise_std: 0.0

train:
  batch_size: 64
  epochs: 25
  lr: 3.0e-4
  weight_decay: 1.0e-4
  patience: 6                          # early stopping on val macro-F1
  weighted_loss: true                  # w_r = N / (K * n_r)
  num_workers: 2
  amp: true                            # mixed precision, ~2x faster on GPU
  out_dir: runs_pretrained

models:
  triage:     {pretrained: true, size: [160, 96], dropout: 0.2}
  family:     {pretrained: true, size: [160, 96], dropout: 0.3}
  fine:       {pretrained: true, size: [160, 96], dropout: 0.3}
  flat:       {pretrained: true, size: [160, 96], dropout: 0.3}
```

**Listing A.3: configs/taxonomy.yaml (semantic class hierarchy)**

```
# Class hierarchy for Cascaded-SolarNet.
#
# Stage 1 separates `healthy` from everything else (early exit).
# Stage 2 routes a defect to one of the families below.
# Stage 3 runs that family's fine-grained CNN.
#
# Label strings must match `anomaly_class` in InfraredSolarModules'
# module_metadata.json exactly. Regroup classes here; no code changes needed.

healthy: No-Anomaly

families:
  Electrical:
    - Cell
    - Cell-Multi
    - Diode
    - Diode-Multi
    - Hot-Spot
    - Hot-Spot-Multi
  Environmental:
    - Soiling
    - Shadowing
    - Vegetation
  Structural:
    - Cracking
    - Offline-Module
```

# APPENDIX B

# KEY SOURCE CODE

The listings show the three pieces of code that define the behaviour of the system: the stage network and the cascade routing, the augmentation, and the attribution of errors to stages. The complete code is available in the project repository.

**Listing B.1: Stage network and cascade routing (src/solarnet/models.py)**

```
class StageCNN(nn.Module):
    """[Conv3x3-BN-ReLU-MaxPool2] x len(channels) -> GlobalAvgPool -> Dropout -> FC.

    With the native 40x24 input, three blocks leave a 5x3 map before pooling.
    """

    def __init__(self, channels: list[int], num_classes: int, dropout: float = 0.0,
in_ch: int = 1,
                 convs_per_block: int = 1):
        super().__init__()
        layers, prev = [], in_ch
        for ch in channels:
            for _ in range(convs_per_block):
                layers += [
                    nn.Conv2d(prev, ch, 3, padding=1, bias=False),
                    nn.BatchNorm2d(ch),
                    nn.ReLU(inplace=True),
                ]
                prev = ch
            layers.append(nn.MaxPool2d(2))
        self.features = nn.Sequential(*layers)
        self.head = nn.Sequential(nn.Flatten(), nn.Dropout(dropout), nn.Linear(prev,
num_classes))
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode="fan_out", nonlinearity="relu")

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        return self.head(F.adaptive_avg_pool2d(x, 1))
```

**Listing B.2: Cascade inference with early exit (src/solarnet/models.py)**

```
    def predict(self, x: torch.Tensor, threshold: float = 0.5, tta: bool = False) ->
dict[str, torch.Tensor]:
        """Route a batch through the cascade (hard routing, with early exit).

        `threshold` is the triage P(defect) above which an image continues to stage 2.
        Returns global class predictions plus the per-stage decisions, so
        evaluation can attribute errors to the stage that made them.
        """
        n = x.shape[0]
        pred = torch.full((n,), self.healthy_index, dtype=torch.long, device=x.device)
        fam_pred = torch.full((n,), -1, dtype=torch.long, device=x.device)

        is_defect = self._probs(self.triage, x, tta)[:, 1] > threshold
        idx = is_defect.nonzero(as_tuple=True)[0]
        if idx.numel():  # early exit: healthy panels never reach stage 2
            fam_pred[idx] = self._probs(self.family, x[idx], tta).argmax(1)
            for f_i, fam in enumerate(self.family_names):
```

```
                sub = idx[fam_pred[idx] == f_i]
                if sub.numel():
                    pred[sub] = self.class_offsets[fam] + self._probs(self.fine[fam],
x[sub], tta).argmax(1)
        return {"pred": pred, "is_defect": is_defect, "family": fam_pred}

    @torch.no_grad()
```

**Listing B.3: Training augmentation (src/solarnet/common.py)**

```
def augment(x: torch.Tensor, aug: dict) -> torch.Tensor:
    """Stochastic train-time augmentation on a [B,1,H,W] batch in [0,1].

    90-degree rotations from the data report are left out: the images are
    40x24, so a quarter turn would change the shape. Two flips together
    already give the 180-degree rotation.
    """
    b = x.shape[0]
    flip_h = torch.rand(b, device=x.device) < aug["hflip"]
    x = torch.where(flip_h[:, None, None, None], x.flip(-1), x)
    flip_v = torch.rand(b, device=x.device) < aug["vflip"]
    x = torch.where(flip_v[:, None, None, None], x.flip(-2), x)

    def jitter(scale: float) -> torch.Tensor:
        return 1 + (torch.rand(b, 1, 1, 1, device=x.device) * 2 - 1) * scale

    x = x * jitter(aug["brightness"])
    mean = x.mean(dim=(1, 2, 3), keepdim=True)
    x = (x - mean) * jitter(aug["contrast"]) + mean
    x = x + torch.randn_like(x) * aug["noise_std"]
    return x.clamp(0, 1)
```

**Listing B.4: Attribution of errors to stages (src/solarnet/evaluate.py)**

```
def stage_errors(y: np.ndarray, out: dict, tax: Taxonomy) -> dict:
    """Attribute every cascade mistake to the first stage that went wrong."""
    healthy = tax.index(tax.healthy)
    true_defect = y != healthy
    true_fam = np.array([tax.family_names.index(tax.family_of(tax.classes[c])) if c !=
healthy else -1 for c in y])
    is_defect = out["is_defect"].numpy()
    fam = out["family"].numpy()
    wrong = out["pred"].numpy() != y

    triage_miss = true_defect & ~is_defect           # defect exited early as healthy
    triage_false_alarm = ~true_defect & is_defect    # healthy sent down the cascade
    family_wrong = true_defect & is_defect & (fam != true_fam)
    fine_wrong = wrong & ~triage_miss & ~triage_false_alarm & ~family_wrong
    return {
        "stage1_missed_defects": int(triage_miss.sum()),
        "stage1_false_alarms": int(triage_false_alarm.sum()),
        "stage2_wrong_family": int(family_wrong.sum()),
        "stage3_wrong_subtype": int(fine_wrong.sum()),
        "total_errors": int(wrong.sum()),
        "early_exit_rate": float((~is_defect).mean()),
    }
```

**Listing B.5: Class weights for the imbalanced loss (src/solarnet/data.py)**

```
def class_weights(targets: np.ndarray, num_classes: int) -> np.ndarray:
    """w_r = N / (K * n_r); classes absent from `targets` get weight 0."""
    counts = np.bincount(targets, minlength=num_classes).astype(np.float64)
```

```
    with np.errstate(divide="ignore"):
        w = np.where(counts > 0, len(targets) / (num_classes * counts), 0.0)
    return w.astype(np.float32)
```

# APPENDIX C

# RUNNING THE DEMONSTRATIONS

Two demonstrations are provided. Both need the trained checkpoints and the dataset, which is downloaded by a script from the public repository of the dataset.

- **Command-line demonstration.** python scripts/predict.py --sample 5 draws, for random test images, the decision and confidence of each stage and saves the figure. The option --class Hot-Spot restricts the sample to one class and --image path loads any image.
- **Browser demonstration.** python scripts/make_demo_html.py creates a single HTML page with three test images per class. Opening it in a browser shows the selected image, and pressing Run cascade reveals Stage 1, Stage 2 and Stage 3 one after another, with the final prediction in green if it is correct and red if not.
- **Reproducing the results.** python scripts/make_splits.py creates the split, python -m solarnet.train --node all --config configs/pretrained.yaml trains the five networks, and python -m solarnet.evaluate --flat --config configs/pretrained.yaml evaluates the cascade and the flat baseline.

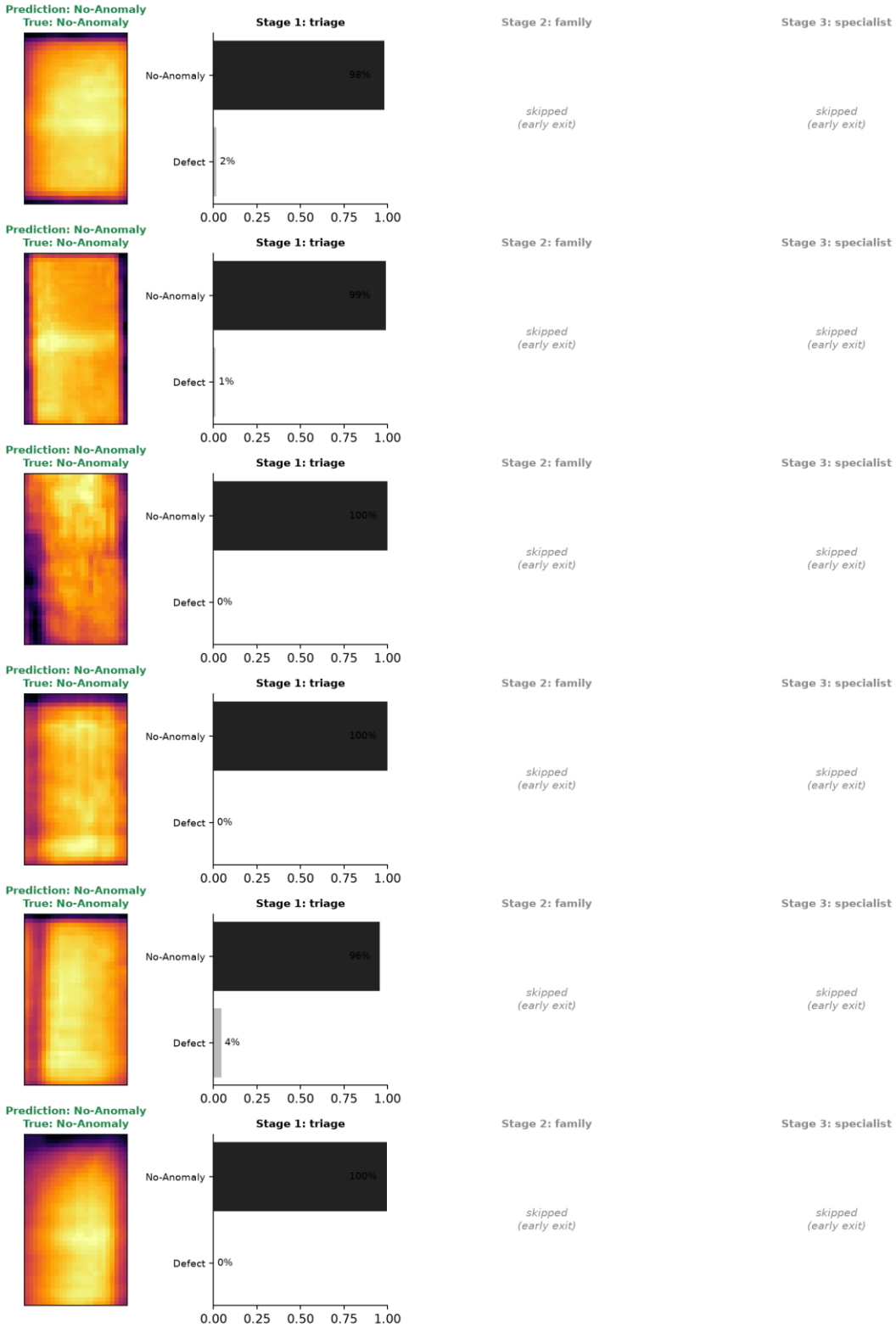



**Fig. 15. Example output of the command-line demonstration for random test images (green: correct, red: wrong).**

# APPENDIX D

# REPOSITORY STRUCTURE

**Table D.1: Layout of the project repository**

| Path | Content |
|---|---|
| src/solarnet/ | Package: taxonomy, data, models, common helpers, training, evaluation |
| configs/ | Experiment configurations and the class hierarchies (semantic and data-driven) |
| scripts/ | Download and split scripts, baselines, cross-validation, figure and report generators, demonstrations |
| tests/ | Automated tests that run on synthetic data |
| results/ | Raw metrics of every reported experiment (JSON) |
| checkpoints/ | Trained Stage 3 specialists shared with the team |
| docs/ | Architecture diagrams and the reports |
| notebooks/ | Notebook that runs the whole pipeline on Colab |